

Enabling Branching Support in Server-Side Vitruvius

Bachelor Thesis of

Linh Nguyen Phuong

At the KIT Department of Informatics
KASTEL – Institute of Information Security and Dependability

First examiner: Prof. Dr.-Ing. Anne Koziolk

Second examiner: Prof. Dr. Ralf Reussner

First advisor: Dr. Raziye Dehghani

Second advisor: Dr. rer. nat Arne Lange

Third advisor: M.Sc. Martin Armbruster

26.01.2026 – 26.05.2026

Karlsruher Institut für Technologie
Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

Enabling Branching Support in Server-Side Vitruvius (Bachelor Thesis)

I declare that I have developed and written the enclosed thesis completely by myself. I have not used any other than the aids that I have mentioned. I have marked all parts of the thesis that I have included from referenced literature, either in their original wording or paraphrasing their contents. I have followed the by-laws to implement scientific integrity at KIT.

Karlsruhe, 26.05.2026



.....
(Linh Nguyen Phuong)

Abstract

Vitruvius is a framework for view-based model-driven software development that maintains a Virtual Single Underlying Model (vsUM), which serves as a unified representation of all design artefacts and enforces consistency across associated views through declarative and imperative propagation rules. In the current architecture, all model changes are applied sequentially to a single global vsUM state. This single-evolution constraint prevents developers from pursuing independent lines of work in parallel, making it impossible to experiment safely, develop features in isolation, or conduct concurrent multi-developer modeling activities. Consequently, the framework lacks a fundamental capability that version control systems have long provided for source code: the ability to branch, isolate, and later reconcile diverging development trajectories. This thesis designs and implements a branch-aware extension to the Vitruvius server that introduces branching as a first-class operation on the vsUM. By leveraging Git as a storage backend, the extension is structured around four interdependent subsystems. The branch lifecycle subsystem provides explicit management of branch operations and parent-child tracking, with metadata persisted so that branch context survives checkout operations. The semantic change recording subsystem captures model modifications as typed, UUID-keyed entries, enabling fine-grained traceability. The merge handling subsystem uses these changelogs to detect and classify semantic conflicts before any file-level merge is attempted, supporting both automatic and manual resolution. Finally, the model state versioning subsystem anchors named snapshots to Git tags to provide a controlled rollback mechanism. These operations are exposed through a REST API to facilitate integration with external tooling. We evaluate the extension against six stated goals through a functional test suite and a case study in the vehicle braking system domain. The results confirm that all goals are met, demonstrating that branch-aware model versioning can be integrated into Vitruvius without compromising its core consistency guarantees.

Zusammenfassung

Vitruvius ist ein Framework für die sichtbasierte, modellgetriebene Softwareentwicklung, das ein zentrales virtuelles Gesamtmodell (VSUM - Virtual Single Underlying Model) als einheitliche Repräsentation aller Entwurfsartefakte verwaltet. Konsistenz zwischen den zugehörigen Sichten wird dabei durch deklarative und imperative Propagierungsregeln sichergestellt. In der aktuellen Architektur werden sämtliche Modelländerungen sequenziell auf einen einzigen globalen vsum-Zustand angewendet. Diese Einschränkung auf eine lineare Evolution verhindert, dass Entwickler unabhängige Arbeitszweige parallel verfolgen können, was das sichere Experimentieren, die isolierte Entwicklung von Features oder die gleichzeitige Modellierung durch mehrere Entwickler unmöglich macht. Folglich fehlt dem Framework eine grundlegende Fähigkeit, die Versionskontrollsysteme für Quellcode bereits seit langem bieten: die Möglichkeit, Entwicklungsverläufe zu verzweigen, zu isolieren und später wieder zusammenzuführen. Diese Arbeit entwirft und implementiert eine Branch-fähige Erweiterung für den Vitruvius-Server, die Branching als „First-Class-Operation“ auf dem vsum einführt. Unter Verwendung von Git als Storage-Backend gliedert sich die Erweiterung in vier voneinander abhängige Teilsysteme. Das Branch-Lifecycle-Subsystem ermöglicht die explizite Verwaltung von Branch-Operationen sowie die Nachverfolgung von Parent-Child-Beziehungen, wobei Metadaten persistent gespeichert werden, sodass der Branch-Kontext über Checkout-Operationen hinweg erhalten bleibt. Das Subsystem zur Aufzeichnung semantischer Änderungen erfasst Modellmodifikationen als typisierte, mit UUIDs versehene Einträge und ermöglicht so eine feingranulare Rückverfolgbarkeit. Das Merge-Handling-Subsystem nutzt diese Änderungsprotokolle, um semantische Konflikte zu erkennen und zu klassifizieren, bevor ein Merge auf Dateiebene versucht wird, und unterstützt sowohl die automatische als auch die manuelle Konfliktlösung. Schließlich verknüpft das Subsystem zur Versionierung des Modellzustands benannte Snapshots mit Git-Tags, um einen kontrollierten Rollback-Mechanismus bereitzustellen. Sämtliche Operationen werden über eine REST-API bereitgestellt, um die Integration mit externen Werkzeugen zu erleichtern. Die Erweiterung wurde anhand von sechs definierten Zielen durch eine funktionale Testsuite und eine Fallstudie im Bereich von Fahrzeugbremsystemen evaluiert. Die Ergebnisse bestätigen, dass alle Ziele erreicht wurden, und zeigen, dass eine Branch-fähige Modellversionierung in Vitruvius integriert werden kann, ohne dessen Konsistenzgarantien zu beeinträchtigen.

Contents

Abstract	i
Zusammenfassung	iii
1 Motivation	1
1.1 Introduction	1
1.2 Goal of thesis	2
1.3 Structure of this thesis	2
2 Foundation	3
2.1 Model-Driven Software Development	3
2.2 View-Based Model-Driven Software Development	3
2.2.1 Models, Metamodels, and View Integration	3
2.2.2 Deltas and Consistency Preservation	4
2.2.3 Framework Architecture and Integration Patterns	4
2.3 Versioning and Branching in Model-Driven Software Development	5
2.3.1 Foundations of Model Versioning in MDSD	5
2.3.2 State-Based Versioning Approaches	5
2.3.3 Operation-Based Versioning Approaches	6
2.3.4 Core Activities in Model Versioning	6
2.3.5 Branching in Model-Driven Software Development	7
2.3.6 Git and JGit	7
2.3.7 Requirements of Model Versioning and Branching Strategy	7
3 Implementation	9
3.1 Requirements	9
3.2 Running Example	11
3.3 System Overview	13
3.3.1 Component Responsibilities	13
3.3.2 Branch Metadata	15
3.3.3 Directory Structure	16
3.3.4 Hook-Watcher Pattern	17
3.4 Git Hook Integration	17
3.5 Branch-Aware Virtual Model	20
3.5.1 Decorator Design	20
3.5.2 Per-Branch Storage Layout	20
3.5.3 In-Place Reinitialization	21

3.5.4	Initialization and Component Orchestration	22
3.5.5	VSUM State Versioning	23
3.6	Branch Context Switching	25
3.6.1	Trigger File Protocol	25
3.6.2	VsumReloadWatcher	25
3.6.3	Race Condition Avoidance	26
3.7	Model Consistency Validation	26
3.7.1	Validation Flow	27
3.7.2	Severity Model	27
3.7.3	Validation Checks	27
3.7.4	Fail-Closed Policy	28
3.8	Semantic Change Recording	29
3.8.1	Architecture	29
3.8.2	Two Commit Paths with Identical Output	30
3.8.3	Reaction Change Filtering	31
3.8.4	Change Classification	32
3.8.5	Changelog Format	33
3.8.6	Change Replay and Conflict Detection	35
3.9	Merge Handling	38
3.9.1	Merge Scenario Classification	38
3.9.2	Implemented Merge Pipeline	39
3.10	Versioning	42
3.11	REST API	43
3.11.1	Endpoint Design	44
3.11.2	Visualization and Version Browsing	44
4	Related Work	47
4.1	Model Comparison and Versioning Frameworks	47
4.2	Pointer-Based File Storage: Git	51
4.3	Comparison of Approaches	51
5	Evaluation	53
5.1	Evaluation Goals and Research Questions	53
5.2	Functional Tests	55
5.2.1	Branch Management and Commit Operations	55
5.2.2	Changelog Storage and Change Recording	55
5.2.3	Event Handlers and Background Watchers	56
5.2.4	File System Utilities and Git Hook Installation	56
5.2.5	Conflict Detection and Merge Handling	57
5.3	Case Study	58
5.3.1	Case Study Domain	58
5.3.2	Test Setup and Methodology	58
5.3.3	Scenario Results	59
5.4	Threats to Validity	61

6	Conclusion and Future Works	63
6.1	Conclusion	63
6.2	Limitations	63
6.3	Future Work	65
	Bibliography	67

1 Motivation

Collaborative development in model-driven engineering raises consistency and coordination challenges that single-threaded model evolution cannot address. This chapter motivates the need for server-side branching in Vitruvius by situating these challenges within the broader context of parallel model evolution and multi-developer workflows. We first examine the limitations of the current Vitruvius, then state the research goal and scope of this thesis, and close with an outline of the remaining chapters.

1.1 Introduction

Modern software development increasingly relies on model-driven engineering (MDE) techniques, where software systems are represented and managed through multiple interrelated models. These models capture different viewpoints, abstraction levels, and architectural perspectives. Ensuring their consistency during evolution is challenging, particularly in collaborative environments where multiple developers may work in parallel. Vitruvius, a framework for view-based software development, addresses this challenge by providing an infrastructure that treats a set of heterogeneous models as a unified Virtual Single Underlying Model (vsuM) [16]. Through this abstraction, developers can work directly on the modeling views that are most appropriate for their tasks, while the framework ensures that the different models remain synchronized, coherent, and consistent. This is achieved through a combination of declarative and imperative consistency preservation rules that propagate changes across views as deltas, enabling a flexible and extensible approach to multi-view model management.

The current Vitruvius, though sophisticated in delta propagation and consistency management, operates on a single linear evolution of the vsuM. All changes are applied sequentially to this global state, preventing multiple independent development lines. In collaborative or large-scale modeling environments, this approach limits experimentation, parallel feature development, and concurrent work, reducing the framework’s scalability and applicability.

Branching, a fundamental concept in modern version control systems, allows multiple isolated workstreams to evolve independently. Native server-side branching in Vitruvius enables developers to manage divergent versions of the vsuM safely, supporting parallel work while preserving consistency. Branching also lays the foundation for essential versioning operations such as merging, conflict detection, and conflict resolution.

1.2 Goal of thesis

This thesis closes this gap by designing and implementing server-side branching support for Vitruvius. The implementation provides explicit branch management, covering the creation, switching, deletion, and tracking of parent-child relationships between branches. It further introduces efficient per-branch delta computation and storage through a semantic changelog mechanism that records model changes at the element level. Finally, it provides a safe merge infrastructure with conflict detection and resolution capabilities that operate at the semantic level, independently of the underlying file format. By extending Vitruvius in this way, the framework gains the ability to support collaborative, scalable, and parallel model-driven development, bridging the gap between its strong consistency mechanisms and the practical requirements of multi-developer engineering workflows.

We evaluate the approach using the Goal-Question-Metric (GQM) framework[8], deriving six evaluation questions from the overarching goal. We address these questions through a functional test suite that verifies each subsystem in isolation, and through a case study conducted in the vehicle braking system domain. The case study exercises four representative collaboration scenarios covering clean merge, conflict detection and resolution, versioning and rollback, and branch lifecycle management. The results confirm that the system correctly handles all four scenarios while preserving bidirectional model consistency throughout.

1.3 Structure of this thesis

This thesis is structured as follows. The present chapter introduces the motivation and research context, outlines the limitations of the current Vitruvius server, and states the goals of the work. Chapter 2 establishes the conceptual foundations, covering model-driven software development, view-based consistency management in Vitruvius, and the principles of versioning and branching in model-driven development. Chapter 3 presents the design and implementation of the branch-aware vsUM extension, including the requirements, the running example used throughout the chapter, the system architecture, and the technical realisation of each subsystem. Chapter 4 surveys related work on model comparison, versioning, and conflict detection, and positions the approach of this thesis relative to existing solutions. Chapter 5 evaluates the implementation using the Goal-Question-Metric framework, reporting the results of the functional test suite and the case study conducted in the vehicle braking system domain. Chapter 6 concludes the thesis with a summary of the contributions and an outline of directions for future work.

2 Foundation

To understand the problem domain and the technical context of the thesis, the relevant conceptual and technical foundations will be introduced, including model-driven software development, view-based consistency management, and model versioning and branching approaches.

2.1 Model-Driven Software Development

Model-Driven Software Development (MDSD) treats models as primary artifacts throughout the software development process. Models capture different system aspects at various abstraction levels, enabling automated transformations, code generation, and early validation of design properties. By raising the abstraction level, MDSD supports more systematic reasoning about system behavior and improves communication among stakeholders. At the same time, the use of multiple heterogeneous models introduces challenges: different views may evolve independently, inconsistencies can arise easily, and maintaining semantic coherence requires dedicated mechanisms for synchronization and coordinated evolution. Addressing these challenges is essential for applying MDSD effectively in collaborative development settings[3].

2.2 View-Based Model-Driven Software Development

This section examines the view-based paradigm in model-driven software development and introduces Vitruvius as the primary technical context for this thesis. It covers the foundational concepts of view integration, the role of deltas in incremental change propagation, and the modular framework architecture that provides the extension points on which the branching support of this thesis is built.

2.2.1 Models, Metamodels, and View Integration

View-based development decomposes a complex system into multiple models, each representing a distinct concern such as structure, behavior, deployment, or performance. While these models focus on different aspects, they frequently overlap, sharing elements that must

remain consistent across views. General frameworks address this by conceptually integrating models into a coherent structure, allowing developers to edit each view independently while maintaining global system semantics[7][26].

Vitruvius exemplifies this approach through the *Virtual Single Underlying Model (vsUM)*[16], which provides a logical unification of all system models. Developers interact with task-specific views, while the vsUM ensures that underlying models remain synchronized. This abstraction hides the complexity of cross-view consistency and provides a foundation for supporting incremental changes and collaborative modeling.

2.2.2 Deltas and Consistency Preservation

Changes in a model are typically represented as *deltas*, capturing additions, deletions, or modifications relative to a previous state. Delta-based processing allows frameworks to propagate changes incrementally, updating only affected parts of the model and reducing computational overhead[15][11]. Consistency preservation ensures that interdependent models remain semantically coherent. This is generally achieved through declarative rules, which define invariants that must hold across models, and imperative rules, which determine how changes should propagate to satisfy these invariants.

In Vitruvius, every change in a view generates a delta, which is propagated to other views according to both declarative and imperative rules[16]. Correspondence models explicitly link related elements across views, enabling precise and efficient updates. This combination of deltas and consistency rules allows incremental evolution while guaranteeing that all views remain semantically consistent.

2.2.3 Framework Architecture and Integration Patterns

View-based frameworks generally separate three key concerns: view definitions that provide task-specific abstractions of the system, change management that propagates updates across views, and consistency preservation that enforces invariants and synchronizes related model elements.

Vitruvius implements these principles through its modular architecture: views provide projections of the vsUM, delta propagation manages updates, and correspondence models maintain element links across views[16]. This separation of concerns ensures extensibility and flexibility. Furthermore, the modular design creates natural extension points for additional features such as server-side branching, where each branch could maintain an independent vsUM, delta history, and correspondence state while reusing the existing view-based editing and consistency mechanisms.

2.3 Versioning and Branching in Model-Driven Software Development

Versioning is a fundamental activity in modern software development, enabling teams to track evolution, manage parallel development, and ensure the recoverability of artifacts. While established textual version control systems such as Git operate on line-based representations, these techniques cannot be directly transferred to MDS. Models are structured, typed graphs that must conform to metamodels and frequently include cross-references and well-formedness constraints[21]. As a consequence, dedicated model versioning techniques have emerged to address the structural and semantic complexity inherent in model artifacts. This section introduces foundational concepts of model versioning in MDS, outlines established versioning approaches as identified in the literature, and discusses the general requirements that influence the suitability of a versioning strategy. These insights form the basis for evaluating an appropriate design for server-side model versioning and branching in the Vitruvius framework.

2.3.1 Foundations of Model Versioning in MDS

Model versioning refers to the processes, algorithms, and representations used to capture, compare, and integrate different versions of models[2]. In contrast to textual artifacts, model changes cannot easily be expressed through line-based differences, as the order of elements in serialized formats such as XMI is not semantically meaningful[20]. Instead, model versioning techniques must operate on the structural and semantic properties of models defined by their metamodels[5]. Two central challenges arise. First, identifying corresponding elements across versions requires robust matching strategies, typically relying on unique identifiers, name heuristics, or structural similarities. Second, model versioning must preserve the well-formedness and consistency of models throughout the versioning lifecycle. This includes respecting metamodel constraints, handling cross-model references, and avoiding the introduction of invalid model states during and also after merging[5].

2.3.2 State-Based Versioning Approaches

State-based versioning approaches operate by comparing two snapshots of a model to compute their differences. The comparison process usually involves a matching phase that aligns corresponding model elements, followed by a differencing phase that identifies added, removed, or modified elements. A representative framework is EMF Compare[6], which provides two-way and three-way diff and merge capabilities for EMF-based models. State-based approaches offer several advantages: they can be applied independently of the modeling editor, and they often provide mature user interfaces for inspecting and resolving differences. Their primary limitations arise from computational cost, as matching

and differencing may be expensive for large or deeply nested models. Moreover, state-based approaches typically operate at a syntactic level, meaning that higher-level, semantic changes (e.g., refactorings or composite operations) may not be reflected accurately[5].

2.3.3 Operation-Based Versioning Approaches

Operation-based versioning approaches record the editing actions performed during model evolution. Instead of comparing full model states, these approaches capture fine-grained change operations such as creating, deleting, or modifying elements. EMFStore is a prominent example, maintaining an operation log that can be replayed or merged to derive new model versions[17]. Operation-based versioning offers several benefits. Deltas remain compact, as only the operations themselves are stored, expensive diff computations are avoided, and the recorded operations often reflect the developer's intent more accurately. However, these systems may become dependent on specific modeling editors or interaction patterns, since the recorded operations must correspond to the semantics of the underlying editor. Handling composite operations is also challenging: if a user-level action spans multiple low-level operations, it must be represented atomically to avoid invalid or inconsistent intermediate states during merging[5].

2.3.4 Core Activities in Model Versioning

Independent of whether a versioning approach is state-based or operation-based, the versioning process is often described as consisting of these fundamental activities[5][19]:

- **Operation detection:** determining operations applied on model versions, either as atomic operation or as a composite of multiple atomic operations.
- **Matching:** identifying corresponding model elements across versions using identifiers or structural heuristics.
- **Differencing:** determining changes between versions, either as low-level operations or as state-based modifications.
- **Conflict Detection:** identifying situations in which concurrent changes cannot be combined without violating model consistency.
- **Conflict Resolution:** providing automated or manual strategies for selecting which changes should take precedence.
- **Merging:** integrating changes into a unified model version, typically using three-way merge strategies.

While syntactic conflicts are commonly addressed, semantic conflict detection remains a limited capability and is still regarded as an open research challenge[7]. Only a small number of approaches, such as SMOVER or domain-specific conflict detectors, address semantic inconsistencies explicitly.

2.3.5 Branching in Model-Driven Software Development

Branching enables parallel development by allowing multiple, isolated sequences of changes to coexist until they are eventually integrated. In textual version control, branching is a well-established concept supported by dedicated primitives such as *branch creation*, *switching*, and *merging*. However, explicit branching support is largely absent or rudimentary in most existing model versioning systems. Several reasons might contribute to this gap. First, model merges typically rely on complex matching and differencing, making repeated merges across multiple diverging branches costly. Second, model semantics and cross-references introduce additional constraints that can lead to conflicts that do not exist in textual artifacts. Third, most versioning systems in the MDSO ecosystem assume linear or centralized repositories with limited support for long-lived or parallel branches. These challenges are magnified in the context of view-based model development. When multiple views represent different projections of a holistic system, branching must ensure that cross-view consistency is preserved across all branches. Versioning must therefore operate not only on isolated model states but on interdependent model deltas and synchronization rules[20].

2.3.6 Git and JGit

Git is a distributed version control system that stores repository state as an immutable object store of commits, trees, and blobs, and manages branch references as lightweight pointers into that store. Because Git operates on arbitrary file content rather than on any specific artifact type, it provides a generic persistence and history layer that model versioning systems can reuse without modifying the core repository format. JGit is the pure-Java implementation of the Git core library, maintained under the Eclipse Foundation. It provides programmatic access to all Git operations, including commit creation, branch and tag resolution, object-store traversal, index manipulation, and three-way file merging. Unlike shell-based Git interaction, JGit executes within the same Java Virtual Machine as the host application, eliminating process-spawning overhead and granting the application direct access to repository internals. The implementation described in this thesis uses JGit for all operations that read from or write to the Git object store, including commit graph traversal via `RevWalk`, historical file retrieval via `TreeWalk`, merge base computation via `RevFilter.MERGE_BASE`, and three-way file-level merging via the recursive merge strategy.

2.3.7 Requirements of Model Versioning and Branching Strategy

To integrate branching and versioning into a model-driven framework, a solution must satisfy a set of general capabilities that are independent of any specific metamodel or tooling environment[5]. At the most fundamental level, the approach must support diverging development lines with efficient merge mechanisms. Change recording must capture fine-grained deltas, composite operations, and semantically meaningful modifications so that the stored history reflects developer intent rather than arbitrary file differences. Conflict detection

must operate at both the syntactic and semantic levels, since standard file-level merge tools cannot identify violations of model well-formedness or cross-view consistency rules. Throughout evolution and merging, the solution must preserve model integrity, including metamodel constraints, cross-model references, and consistency invariants shared across multiple views. The approach must further scale to large models and long variant histories without degrading responsiveness, remain applicable across heterogeneous metamodels and modeling environments, and allow domain-specific conflict handlers and merge strategies to be added without modifying the core. These general requirements serve as the evaluation baseline for the design choices made in this thesis and are translated into concrete Vitruvius-specific requirements in Chapter 3.

3 Implementation

This chapter details the architectural design and technical implementation of the branch-aware model versioning system, developed as an extension of the current Vitruvius framework. To facilitate comprehensive versioning support, the system enhances Vitruvius with four core functionalities: branch-aware vsUM state management, semantic change recording, a specialized merge handling infrastructure as well as a model state versioning subsystem. The following sections describe the technical realization of these capabilities and the interplay between the underlying components.

3.1 Requirements

The implementation addresses a set of concrete functional requirements derived from the thesis objectives for extending Vitruvius with branch-aware model versioning. To keep framework intrusion minimal, each requirement group is realized by a dedicated subsystem that integrates with Vitruvius through well-defined extension points rather than modifying its core. The requirements fall into four functional groups: branch lifecycle management, semantic change recording and version marking, merge infrastructure with conflict handling, and programmatic access via a REST API. The subsequent sections of this chapter describe how each group is realized, providing a traceable connection between the stated requirements and the components that implement them.

Branch Lifecycle Management. The branch lifecycle management group covers the operations necessary to support parallel model development within a shared Vitruvius repository. The system must allow developers to create new branches from any existing branch, associating each with framework-specific metadata that Git does not natively provide. This metadata includes the name of the parent branch, creation and last-modification timestamps, a lifecycle state, and a maturity level. The lifecycle state reflects whether a branch is currently active, has been integrated into another branch, or has been explicitly removed. The maturity level allows teams to annotate branches as draft work, reviewed content, or finalized output[1]. Beyond creation, the system must support listing all branches together with their metadata, switching the active branch while correctly restoring the corresponding vsUM state, explicit deletion, and automatic removal following a successful merge. Parent-child relationships between branches must be persisted and queryable at runtime so that the full branch topology can be inspected programmatically. The system must also reject duplicate branch names to prevent identifier ambiguity.

Semantic Change Recording and Version Marking. The second group covers the recording of model changes at fine-grained semantic granularity and the ability to mark stable model states as named versions. Rather than relying on textual diffs, we require that every atomic change applied to a model element is captured as a typed changelog entry at commit time. Each entry must identify the affected element by its stable Vitruvius UUID, record the specific structural feature involved, preserve the previous and updated values, and classify the change according to its structural operation kind. Crucially, each entry must also annotate whether the change was initiated directly by the developer or generated automatically when Vitruvius triggered the Consistency Preservation Rules (CPRs) that the methodologist had defined. The system must filter framework-generated changes from the recorded set so that the changelog captures only explicit developer intent. The output is a JSON file keyed by element UUID, named after the abbreviated commit SHA, and stored under the versioned changelog directory. This file must be produced regardless of whether the commit originates from the REST API or a direct Git client, with the guarantee that exactly one changelog is produced per commit across both paths. For version management, developers must be able to mark any commit as a named version with an optional description and a maturity classification, and must be able to roll back the working branch to any previously marked version through a two-step confirmation workflow that previews the impact before executing the reset.

Merge Infrastructure and Conflict Handling. The third group covers the integration of diverging branch histories into a consistent model state. The system must support both fast-forward and non-fast-forward three-way merges using the common ancestor of the two branches. Before any file-level merge is attempted, a semantic pre-check must compare the recorded changelogs of both branches at the model-element level using the element UUID assigned by the Vitruvius to each model element, so that conflicts are detected independently of serialisation order or file position. Detected conflicts must be classified by severity: modifications to the same structural feature by both branches are treated as direct conflicts, and a separate detection step must identify tombstone conflicts, which arise when one branch deletes a container element while the other modifies a child of that container. When a conflict involves one developer-initiated change and one framework-generated change triggered by a CPR, the system must automatically resolve the conflict in favour of the developer-initiated change, since the framework-generated change would be reproduced upon replay. When all direct conflicts are resolved, the system must fall back to semantic engine replay if a textual merge conflict arises from structurally independent changes to the same file. For conflicts that require an explicit resolution strategy, the system must support accepting either the source branch version or the target branch version of the affected elements. After a successful merge, the in-memory vsUM must be reloaded from the merged model files, and the source branch must be marked as merged in its metadata.

Programmatic Access via REST API. The fourth group requires that all operations introduced by the preceding three groups are accessible through an HTTP API, so that external tools, automated test harnesses, and IDE integrations can interact with the vsUM without requiring direct Java embedding. Every branch lifecycle operation must be exposed as a dedicated endpoint, including creation, deletion, switching, listing, and querying of lifecycle state

and maturity level. The commit, merge, and conflict resolution operations must equally be accessible over HTTP, with the merge endpoint accepting an optional resolution strategy so that callers can trigger automatic conflict resolution programmatically. The versioning operations, including version creation, retrieval, deletion, rollback preview, rollback confirmation, and branch creation from a named version, must all be reachable via the API. In addition to write operations, the API must expose read endpoints that allow callers to inspect the branch topology, retrieve the per-commit semantic change history of a branch, and query the aggregated semantic delta between a branch and a given base branch since their divergence point. Finally, the existing view access capabilities of the Vitruvius server, covering view type discovery, view selector construction, and view retrieval and update, must be preserved and remain accessible through the same API.

3.2 Running Example

To ground the concepts introduced throughout this chapter, we use a running example drawn from automotive engineering[12]. The example involves two coupled metamodels: a *Brakesystem* metamodel, which captures the structural components of a brake assembly, and a *CAD_Model* metamodel, which stores the corresponding computer-aided design parameters as named numeric values within namespaces. The *Brakesystem* metamodel defines three element types: a *BrakeDisk* with attributes *id*, *diameterInMM*, and *brakeDiskThicknessInMM*; an *ABSSensor* with attributes *id*, *lengthInMM*, and *numberOfPins*; and a *BrakeCaliper* with attributes *id* and *pistonDiameterInMM*. The two metamodels are coupled through bidirectional CPRs defined by the methodologist: changes to brake component attributes on the *Brakesystem* side are automatically propagated to the corresponding numeric parameters in the CAD model, and vice versa. The shared starting state, used as the common ancestor across all scenarios, consists of a single brake disk instance *disk1* with a diameter of 120mm and a thickness of 30mm, committed to the main branch. Two developers then work in parallel on separate branches and produce changes that are later reconciled. We revisit this example in four concrete scenarios that cover non-conflicting parallel changes, semantic conflicts, model versioning with rollback, and branch lifecycle management, and we refer back to it in the Evaluation chapter (Chapter 5) to validate the implemented system against the stated requirements.

The four scenarios are designed to cover complementary aspects of the implemented system and to be reused as reference points in the remaining sections of this chapter. In the first scenario (S1), two developers work on separate branches without modifying the same model element: one developer increases the brake disk diameter on a feature branch, while the other adds an ABS sensor on the main branch. This scenario demonstrates that the system correctly identifies the absence of semantic conflicts and completes the merge automatically. In the second scenario (S2), both developers independently modify the same attribute of the same element, producing a true semantic conflict that the system detects and reports. The scenario further shows how the conflict is resolved by applying an explicit resolution strategy that selects one branch's version of the attribute. In the third scenario (S3), the state

of the main branch is captured as a named version before a structural change is applied; a subsequent rollback operation restores the branch to that captured state, verifying that the versioning mechanism preserves and recovers model content faithfully. In the fourth scenario (S4), a new branch is created, an additional component is added exclusively on that branch, and the branch is then inspected and deleted, confirming that the branch lifecycle operations maintain correct isolation between branches throughout.

Figure 3.1 illustrates the structure of the two coupled metamodels and their consistency relationship. The *Brakesystem* metamodel forms the engineering-facing representation, while the *CAD_Model* metamodel stores the same dimensional values as named numeric parameters grouped into namespaces. The bidirectional CPRs ensure that whenever a developer modifies a brake component attribute through the V-SUM, the corresponding numeric parameter in the CAD model is updated automatically, and vice versa. This tight coupling means that any branch operation, whether a commit, a merge, or a rollback, must preserve consistency across both models simultaneously.

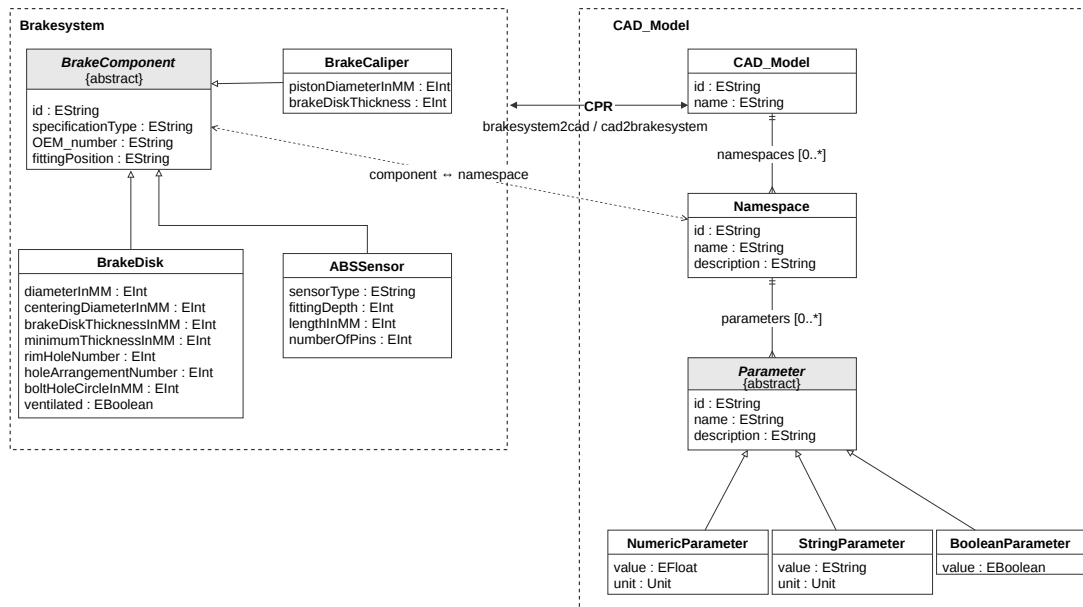


Figure 3.1: Structure of the *Brakesystem* and *CAD_Model* metamodels and their bidirectional consistency preservation relationship.

Before any scenario begins, the system is initialised with a common ancestor state on the main branch. This state contains a single *BrakeDisk* instance with identifier `disk1`, a `diameterInMM` of 120mm, and a `brakeDiskThicknessInMM` of 30mm. Because the CPRs are active from the moment the element is created, the corresponding numeric parameters are simultaneously present in the CAD model. This initial commit serves as the divergence point for the branch-based scenarios S1 and S2, and as the baseline version captured before the modifications in S3. All four scenarios therefore share the same starting condition, which allows the evaluation to isolate and compare the behaviour of individual system features without interference from differing initial states.

3.3 System Overview

The branch-aware model versioning system is architected as a modular suite of collaborating components integrated with the Vitruvius vsUM runtime. The components are organized into six functional domains: core branch management, event watcher infrastructure, logic handlers, semantic changelog recording, merge engine, model versioning, and a REST API layer that exposes all functionality to remote clients. To ensure separation of concerns, communication between the active Java process and the Git hook shell scripts is implemented via a file-based signaling mechanism. Each hook writes a trigger file and terminates immediately, while a background monitoring service detects these files within a defined polling interval and dispatches the event to the corresponding handler. This design ensures that the developer's Git operations remain non-blocking regardless of the processing overhead incurred by the vsUM.

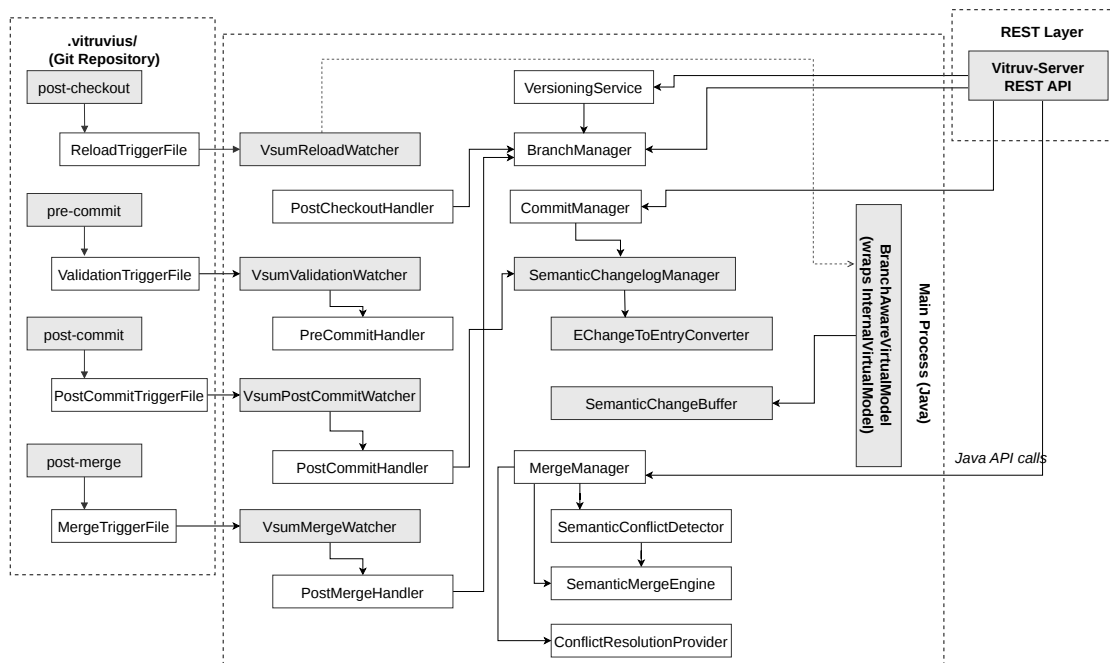


Figure 3.2: High-level component diagram of the branch-aware model versioning system.

3.3.1 Component Responsibilities

The functional responsibilities of the system are distributed across several specialized subsystems. Table 3.1 outlines the core management components and the four Git-hook watcher/handler pairs that form the event-processing layer, while the semantic merging and versioning components are detailed in Table 3.2. Finally, Table 3.3 provides an overview of the REST API layer provided by the Vitruv-Server.

Table 3.1: Core management and event-handling components.

Component	Functional Responsibility
BranchAwareVirtualModel	Decorates the internal virtual model to enable in-place vsUM state restoration during branch transitions.
BranchManager	Orchestrates the branch lifecycle and persists branch metadata.
CommitManager	Coordinates model file staging, Git commit execution, and semantic changelog generation.
MergeManager	Executes the full merge pipeline: fast-forward, semantic pre-check, and engine replay.
VsumReloadWatcher	Polls post-checkout trigger files and forwards branch-switch events to PostCheckoutHandler.
PostCheckoutHandler	Prepares the branch vsUM layout, inherits files from the previous branch, and reinitializes the vsUM in place.
VsumValidationWatcher	Polls pre-commit trigger files and, under a lock, delegates consistency validation to PreCommitHandler.
PreCommitHandler	Validates vsUM integrity: resource loadability, proxy resolution, correspondence-model access, and UUID resolver availability.
VsumPostCommitWatcher	Polls post-commit trigger files, skips system commits, and delegates changelog writing to PostCommitHandler.
PostCommitHandler	Writes the semantic changelog for commits made via the direct Git path and commits it to the branch ref using low-level JGit.
VsumMergeWatcher	Polls post-merge trigger files, delegates post-merge reconciliation to PostMergeHandler, and auto-stages the written merge metadata.
PostMergeHandler	Validates the merged vsUM, copies vsUM state from the source branch, reloads the model, marks the source branch as merged, and writes merge metadata.

Table 3.2: Semantic changelog and versioning subsystem components.

Component	Functional Responsibility
EChangeToEntryConverter	Converts EChange instances into SemanticChangeEntry for serialization.
SemanticChangeBuffer	Accumulates atomic EChange events and filters out reaction-produced changes.
SemanticChangelogManager	Persists semantic change records as JSON documents under <code>.vitruvius/changelogs/</code> .
SemanticConflictDetector	Compares JSON changelogs to identify element-level conflicts by UUID and feature name.
SemanticMergeEngine	Replays source branch changes through the live vsUM to trigger CPRs.
ConflictResolutionProvider	Supplies resolution strategies (THEIRS/OURS) to conflicting element-feature pairs.
VersioningService	Manages Git annotated tags for immutable snapshots and rollback previews.

Table 3.3: Vitruv-Server REST API endpoints and functionalities.

Endpoint Group	Functional Responsibility
Branch Endpoints	Expose creation, deletion, switching, topology, and maturity updates over HTTP.
Commit Endpoint	Accepts model change payloads and triggers server-side commit and changelog generation.
Merge Endpoint	Executes the full remote merge pipeline using a specified source branch and resolution strategy.
Version Endpoints	Provide version retrieval, listing, and the ability to branch from specific snapshots.
Changelog Endpoint	Returns the raw JSON semantic changelog for a specific branch and commit SHA.

3.3.2 Branch Metadata

Each branch is associated with a metadata record persisted as a JSON file at `.vitruvius/branches/<branchName>.metadata`. This file is committed to the Git repository alongside the model files so that it remains visible across developer workstations and repository clones. The record captures five fields: the branch name, the current lifecycle state (ACTIVE, MERGED, or DELETED), the maturity level (DRAFT, REVIEWED, or FINAL), the name of the parent branch from which this branch was created, and the timestamps of creation and last modification. The lifecycle state is updated automatically by the BranchManager when a branch is merged or deleted. The maturity level is a freely settable field exposed through the BranchManager API and the corresponding REST endpoint. Within the scope of this thesis, the maturity

level can be freely set to any of the three values without enforced transitions, serving as the integration point for the accompanying work on maturity-aware branching [1], which extends this mechanism with workflow-specific transition rules and editorial tooling.

3.3.3 Directory Structure

All system-level infrastructure artifacts are encapsulated within a `.vitruvius/` subdirectory at the repository root. This organizational structure ensures a strict separation between internal framework metadata and the developer's model files. Figure 3.3 illustrates the hierarchical layout of the directory.

Directory Structure	Description / Persistence Type
<pre> .vitruvius/ ├── vsum/ │ ├── <branchName>/ │ │ ├── uuid.uuid │ │ ├── models.models │ │ └── correspondences.correspondence │ ├── changelogs/ │ │ ├── <branchName>/ │ │ │ └── json/ │ │ │ └── <shortSha>.json │ ├── branches/ │ │ └── <branchName>.metadata │ ├── merges/ │ │ └── <mergeCommitSha>.metadata │ ├── versions/ │ │ └── <versionId>.metadata │ ├── merge-driver.properties │ ├── reload-trigger │ ├── validate-trigger │ ├── post-commit-trigger │ ├── merge-trigger │ ├── validation-result-<id> │ ├── merge-cache/ │ ├── .reload.lock │ ├── .validation.lock │ └── .amend-in-progress </pre>	<i>versioned: per-branch VSUM state</i> EObject-to-UUID mappings Registered model resource URIs Cross-model trace links <i>versioned: semantic changelog artefacts</i> Human-readable semantic changelog <i>versioned: branch lifecycle metadata</i> <i>versioned: merge operation records</i> <i>versioned: version metadata</i> <i>versioned: Git merge driver config</i> <i>transient: post-checkout signal</i> <i>transient: pre-commit signal</i> <i>transient: post-commit signal</i> <i>transient: post-merge signal</i> <i>transient: pre-commit validation outcome</i> <i>transient: intermediate merge workspace</i> <i>transient: concurrency lock (reload)</i> <i>transient: concurrency lock (validation)</i> <i>transient: amendment guard flag</i>

Figure 3.3: Annotated `.vitruvius/` directory layout.

The directory distinguishes between two categories of data. Versioned artifacts, namely the `vsum/`, `changelogs/`, `branches/`, `merges/` and `versions/` directories together with `merge-driver.properties`, are committed to the version control system. This ensures that the `vsum` state, the semantic change history, and the branch metadata remain synchronized across distributed developer workstations. Transient artifacts serve as inter-process communication signals between Git hooks and the Java process. These files are excluded from Git tracking via a `.gitignore` template managed by the `GitHookInstaller` and are deleted once the corresponding handler has consumed them.

3.3.4 Hook-Watcher Pattern

The interaction between the version control system and the Java-based framework follows a consistent asynchronous pattern. Each Git hook acts as a producer that writes a trigger file to the `.vitruvius/` directory and terminates immediately, ensuring that the developer's Git operations remain non-blocking. On the consumer side, a dedicated background watcher thread monitors the filesystem at a fixed polling interval of 500ms. Upon detecting a trigger file, the watcher performs an atomic read-and-delete operation to prevent duplicate processing and dispatches the event to the appropriate handler. The system implements four such pipelines: the pre-commit hook feeds the `VsumValidationWatcher` and `PreCommitHandler` for model consistency validation, the post-commit hook feeds the `VsumPostCommitWatcher` and `PostCommitHandler` for semantic changelog generation, the post-checkout hook feeds the `VsumReloadWatcher` for `vsum` branch context switching, and the post-merge hook feeds the `VsumMergeWatcher` and `PostMergeHandler` for `vsum` state reconciliation after a merge. An alternative design based on local socket communication was considered but rejected in favour of this file-based approach for its platform independence and ease of debugging. A detailed discussion is provided in Section 3.4.

3.4 Git Hook Integration

Git hooks are shell scripts that Git executes automatically at specific points in the version control lifecycle. The infrastructure integrates with four hook events: pre-commit, post-commit, post-checkout, and post-merge. Each hook is responsible for a single functional concern and is designed to be lightweight. Hooks communicate with the running Java application by writing trigger files to the `.vitruvius/` directory. Background watcher threads on the Java side poll for these files and dispatch the corresponding handling logic. This separation ensures that the developer's Git workflow remains responsive, since the hooks themselves perform no model processing.

Hook installation. The `GitHookInstaller` component manages hook installation programmatically at application startup. Each hook script is bundled as a classpath resource within the application JAR, which ensures that the scripts remain consistent with the Java implementation that processes their output across all deployments. During installation, the

installer writes each script to the `.git/hooks/` directory and sets the necessary executable permissions. If a hook file already exists at the target path, it is first backed up under the name `<hookName>.backup` before the new script is written. The installation is idempotent: `installAllHooks()` checks whether all four hooks are already present and skips the entire installation when they are, making it safe to invoke unconditionally on every startup. In addition to the hook scripts, `installAllHooks()` also updates the repository's `.gitignore` file by appending Vitruvius-specific exclusion entries, or creates the file if it does not exist. For the CLI-driven merge path, the `installAll()` method additionally writes a `.gitattributes` file and a merge driver configuration, which is discussed in Section 3.9.

pre-commit hook. The pre-commit hook executes before Git creates a commit object, giving the infrastructure an opportunity to validate model consistency before the commit is finalized. The hook generates a unique request identifier and writes a trigger file to `.vitruvius/validate-trigger` containing the current commit SHA, the branch name, the request identifier, and a timestamp. It then polls for a per-request result file with a timeout of 30 seconds. If the result file indicates a consistency violation, the hook exits with a non-zero status code, causing Git to abort the commit and display the validation messages to the developer. If validation passes, the hook additionally stages the branch metadata file before allowing the commit to proceed, so that the updated `lastValidated` timestamp is included in the commit. If the polling timeout expires without a result file appearing, the hook also exits with a non-zero status code and blocks the commit with the message *Commit blocked for safety*. This fail-closed behavior is intentional: allowing an unvalidated commit to proceed silently would defeat the purpose of the consistency check. The validation logic executed by the watcher on the Java side is described in Section 3.7.

post-commit hook. The post-commit hook fires after Git has successfully created a commit. Before taking any further action, the hook inspects the set of files changed in the commit and exits immediately if none of them are model files (files with extensions such as `.xml`, `.ecore`, or `.genmodel`). This early exit avoids triggering changelog generation for commits that contain only non-model changes, such as documentation or configuration updates. When model changes are present, the hook first amends the commit to include the current `vsuM` directory for the active branch. It stages the contents of `.vitruvius/vsuM/` and rewrites the commit if any `vsuM` files have changed relative to what was already staged. A guard file prevents the hook from recursively invoking itself during the amend. Once the amend is complete, the hook captures the final commit SHA and writes a trigger to `.vitruvius/post-commit-trigger` containing the SHA, the branch name, a request identifier, and a timestamp. The background `VsuMPostCommitWatcher` and `PostCommitHandler` consume this trigger asynchronously to generate the semantic changelog as described in Section 3.8. For merge commits, which are identified by the presence of a second parent, the hook additionally writes a trigger to `.vitruvius/merge-trigger` so that merge metadata is recorded even when the merge required manual conflict resolution and was finalized through a follow-up commit rather than by the post-merge hook directly.

post-checkout hook. The post-checkout hook executes after every `git checkout` operation. Git supplies three arguments to the hook: the SHA of the previous HEAD, the SHA of the new HEAD, and a flag whose value is 1 for a branch switch and 0 for a file-level checkout. The hook inspects this third argument and exits immediately when no branch transition has occurred, limiting the infrastructure overhead to true branch-switching events. For branch switches, the hook resolves the name of the previous branch by parsing the most recent reflog entry, and writes a trigger to `.vitruvius/reload-trigger` containing the new branch name, a request identifier, a timestamp, and the old branch name. The `VsumReloadWatcher` reads this trigger and initiates a vsUM context switch as described in Section 3.6. The `PostCheckoutHandler` on the Java side also uses the old branch name to initialise the new branch's vsUM storage from the correct parent directory when the branch is encountered for the first time, covering the case where a branch is created and switched to directly via the Git CLI rather than through the REST API.

post-merge hook. The post-merge hook fires after a `git merge` or `git pull` operation completes. The hook distinguishes between two merge outcomes by testing whether a second parent commit exists. A fast-forward merge advances the branch pointer without creating a merge commit, so no second parent is present. In this case the hook resolves the source branch name from the reflog and copies the source vsUM directory directly to the target branch directory in the shell script, then exits. A true merge commit has two parents, indicating that Git combined two divergent histories. For this case the hook resolves the source branch from the second parent commit and writes a trigger to `.vitruvius/merge-trigger` containing the merge commit SHA, the source branch name, the target branch name, a request identifier, and a timestamp. It then polls for a per-request result file for up to 30 seconds and displays the outcome to the developer before exiting. Unlike the pre-commit hook, the post-merge hook always exits with status zero because the merge has already been recorded in Git history and any consistency issues discovered at this point are reported as informational warnings rather than blocking errors.

Design rationale for file-based signaling. The trigger-file mechanism was chosen over direct inter-process communication between the hooks and the Java application. An alternative design using local sockets or named pipes would eliminate the polling latency inherent in the file-based approach and provide a synchronous channel. However, socket-based communication requires the Java process to be active and listening on a predefined address before the hook executes. It also introduces platform-specific configuration complexity, particularly when supporting both native Linux environments and Windows Subsystem for Linux, where the socket namespaces of the two environments are isolated. File-based signalling avoids both constraints: trigger files are written and read using standard filesystem operations that behave identically across all supported platforms, and the hook does not need to know whether the Java application is currently running. The trigger files also serve as a debugging aid, since their presence on the filesystem provides a visible trace of which hook events have fired and have not yet been consumed. The polling interval of 500 milliseconds introduces a maximum latency of one polling cycle between a Git event and the corresponding vsUM reaction, which remains acceptable for an interactive development workflow.

3.5 Branch-Aware Virtual Model

A primary requirement of branch-aware model versioning is that the in-memory `vsuM` instance must consistently reflect the model state of the currently active Git branch. Standard Vitruvius does not support dynamic branch switching: a `VirtualModelImpl` instance is initialized against a fixed storage directory and provides no built-in mechanism to redirect its persistent state at runtime. Modifying `VirtualModelImpl` directly would require invasive changes to the Vitruvius core and violate our constraint of minimal framework intrusion. We address this by applying the Decorator pattern through the `BranchAwareVirtualModel` class, which adds branch-switching capability as an external layer around the existing implementation without touching the core.

3.5.1 Decorator Design

`BranchAwareVirtualModel` implements the `InternalVirtualModel` interface, the same interface used by all watchers, handlers, and the application entry point. The decorator holds a single `InternalVirtualModel` instance as its active model and forwards every interface method to it, making the decorator entirely transparent to existing callers. Those components continue to operate against the `InternalVirtualModel` reference they receive at construction time and remain unaware that a decorator backs it rather than a raw `VirtualModelImpl`.

We add three methods outside the standard interface. `switchBranch(oldBranch, newBranch)` performs the branch context switch described in Section 3.5.3, and `getActiveBranch()` returns the name of the currently loaded branch. Only the `VsumReloadWatcher` holds a reference typed as `BranchAwareVirtualModel`, since it is the only component that needs to call `switchBranch()`. All other components interact with the decorator through the `InternalVirtualModel` interface, preserving the principle of least privilege.

A third method, `getChangeBuffer()`, exposes the `SemanticChangeBuffer`. The decorator instantiates this buffer at construction time and registers it as a `ChangePropagationListener` on the inner model, so it accumulates every atomic `EChange` event produced between commits. The buffer is shared with both commit paths for semantic changelog generation, as described in Section 3.8.

3.5.2 Per-Branch Storage Layout

We assign an independent `vsuM` storage directory to each Git branch at `.vitruvius/vsuM/`. This directory holds the UUID mappings (`uuid.uuid`), the correspondence model (`correspondences.correspondence`), the model names file (`models.models`), and the consistency metadata subfolder, all specific to the model state of that branch. The `VsumFileSystemLayout` component manages this per-branch organisation. We extended it with a static factory method `forBranch(repoRoot, branchName)` that constructs a layout instance for a given

branch name without invoking JGit to resolve the current HEAD. This bypass is necessary during branch switching because JGit's HEAD may not yet reflect the newly checked-out branch at the moment `switchBranch()` is invoked by the asynchronous watcher.

When a branch is checked out for the first time, its vsun directory does not yet exist. In this case `inheritFromBranchIfEmpty()` copies the entire source branch vsun directory to the new branch, including the UUID mappings, the correspondence model, the model names file, and the consistency metadata. Copying only the UUID and correspondence files would leave `models.models` absent, which prevents the UUID resolver from loading model objects and causes resolution failures on the first model access. After the copy, we rebase the relative cross-reference paths inside the copied correspondence file so that they are expressed relative to the target branch directory rather than the source. This step is required when the two branch names sit at different directory depths, since EMF serialises cross-resource references as paths relative to the containing file and an unadjusted copy would cause proxy resolution to fail silently.

3.5.3 In-Place Reinitialization

Branch switching is implemented as an in-place reinitialization of the active model rather than a dispose-and-recreate cycle. When `switchBranch(oldBranch, newBranch)` is invoked, the method first constructs a `VsunFileSystemLayout` for the target branch using `forBranch()`, bypassing JGit HEAD resolution as described above. It then calls `prepare()` to create the target branch directory on disk and `inheritFromBranchIfEmpty()` to populate it if this is the first time the branch is accessed. Before delegating to the inner model, we drain the `SemanticChangeBuffer` to discard any accumulated `EChange` events that reference the current branch's `EObject` instances. These references become unresolvable once the resource set is unloaded, so they must be cleared before reinitialization begins. The decorator then calls `reinitialize(newLayout)` on the wrapped model, which redirects the file system layout, recreates the correspondence model bound to the target branch's file, loads all current resources, resets the UUID resolver, and reloads the model files from the target branch directory. The `activeBranchName` field is updated only after a successful reinitialization, ensuring it always reflects the last known valid state even if the operation fails partway through.

We evaluated and rejected an alternative approach in which the active model is disposed and a new `VirtualModelImpl` instance is constructed for each branch switch. This is not viable for two reasons. First, the `VirtualModelRegistry` enforces a singleton constraint of one active instance per storage folder, and `dispose()` deregisters the instance without supporting re-registration. Recreating the instance would therefore require a full application rebuild. Second, the consistency preservation rule configuration is supplied through the `VirtualModelBuilder` at construction time and is not persisted to disk. An object-replacement strategy would require this configuration to be re-supplied at every branch switch, incorrectly coupling `BranchAwareVirtualModel` to application-level details outside its scope. The in-place reinitialization avoids both constraints by retaining the registry entry and the rule configuration across all branch transitions.

3.5.4 Initialization and Component Orchestration

At application startup, the inner `VirtualModelImpl` is constructed via the `VirtualModelBuilder` with the full vsumm configuration, including the storage folder, the user interactor, and the consistency preservation specifications. This instance is immediately wrapped in a `BranchAwareVirtualModel` decorator. The decorator resolves the initial branch name by inspecting the current Git HEAD, so `getActiveBranch()` returns the correct value from the first invocation regardless of which branch is active at startup.

The decorated instance is then distributed to the monitoring components. The `VsumReloadWatcher` receives a reference typed as `BranchAwareVirtualModel` because it is the only component that needs to invoke `switchBranch()`. All other watchers receive the reference through the `InternalVirtualModel` interface. Semantic change tracking must be wired before the first commit is processed. We call `attachSemanticChangeTracking()` on both the `CommitManager` and the `VsumPostCommitWatcher`, supplying the shared `SemanticChangeBuffer`, a `Supplier` that fetches the resolver fresh after each reinitialization, and a supplier for the currently loaded model resources. This wiring ensures that whichever commit path fires first drains the shared buffer, while the other finds it empty.

```
1  InternalVirtualModel innerModel = new VirtualModelBuilder()
2      .withStorageFolder(repoRoot)
3      .withUserInteractorForResultProvider(interactor)
4      .withChangePropagationSpecifications(cpSpec)
5      .buildAndInitialize();
6  BranchManager branchManager = new BranchManager(repoRoot);
7  BranchAwareVirtualModel model =
8      new BranchAwareVirtualModel(repoRoot, innerModel, branchManager);
9
10 CommitManager commitManager = new CommitManager(repoRoot);
11 commitManager.attachSemanticChangeTracking(
12     model.getChangeBuffer(),
13     model::getUuidResolver,
14     model::getViewSourceModels);
15
16 VsumPostCommitWatcher postCommitWatcher = new VsumPostCommitWatcher(repoRoot);
17 postCommitWatcher.attachSemanticChangeTracking(
18     model.getChangeBuffer(),
19     model::getUuidResolver,
20     model::getViewSourceModels);
21
22 // VsumReloadWatcher requires the typed BranchAwareVirtualModel reference
23 // to call switchBranch(); all other watchers use InternalVirtualModel.
24 new VsumReloadWatcher(model, repoRoot).start();
25 new VsumValidationWatcher(model, repoRoot).start();
26 new VsumMergeWatcher(model, repoRoot).start();
27 postCommitWatcher.start();
```

Listing 3.1: Startup wiring sequence for the branch-aware vsum.

3.5.5 VSUM State Versioning

The per-branch vsum state must be persisted within the Git repository alongside the corresponding model files so that it remains synchronized across developer workstations and survives repository re-initialization. Without this, the directories under `.vitruvius/vsum/<branchName>/` would remain local artifacts, invisible to collaborators and lost during a re-clone. We achieve this through a commit amendment strategy implemented in the `post-commit` hook, which injects the vsum directory into each model commit retroactively.

Commit Amendment Strategy The vsum state for a given branch is stored as a directory at `.vitruvius/vsum/`, containing the UUID resolver file, the correspondence model, the model names file, and the consistency metadata subfolder, as described in Section 3.5.2. All of these files must be committed together with the model files to ensure that any commit in the repository history carries a self-contained, reconstructable vsum snapshot. Requiring the developer to stage these internal artifacts manually is impractical: it exposes framework internals to the development workflow and is error-prone under time pressure. Instead, the `post-commit` hook injects the vsum directory into the commit retroactively. The hook first checks whether the commit contains any model file changes; if none are present, the hook exits immediately and the amendment is skipped. When model files have changed, the hook stages the entire branch vsum directory and, if the staging produces any change relative to the previous commit, rewrites the commit using `git commit -amend -no-edit -no-verify` to produce a single atomic commit containing both the model files and the corresponding vsum snapshot. If the vsum directory does not exist or no vsum files have changed, the amend step is skipped entirely.

3.5.5.1 Infinite Recursion Prevention

The `--amend` operation causes Git to fire the `post-commit` hook a second time, because Git treats an amendment as a new commit event. Without a guard mechanism, each amendment would trigger another hook execution, producing an infinite cycle. We prevent this using a guard file at `.vitruvius/.amend-in-progress` whose content is the process identifier of the hook invocation that initiated the amend.

Before performing the amend, the hook checks for the presence of this file. If the file does not exist, the hook writes the current shell PID to it immediately before calling `git commit -amend`. If the file exists and the recorded PID corresponds to a live process, the hook concludes that it is the recursive re-entry triggered by the amend and exits immediately after deleting the guard file. If the file exists but the recorded PID is no longer alive, the guard was left by a process that terminated abnormally. In this case the hook discards the stale guard with a warning message and continues normally, so the current commit is not silently left without a vsum snapshot. If the guard file does not exist, the hook proceeds with the normal amendment flow. This PID-based protocol ensures that the amend cycle executes exactly once per developer commit while recovering automatically from abrupt process termination, without requiring any manual intervention from the developer.

3.5.5.2 Repository Configuration

Not all files within the `.vitruvius/` directory should be versioned. Trigger files, lock files, and validation results are transient artifacts. Their contents are meaningful only during a single hook-watcher interaction cycle. Including these files in the repository would produce spurious commits and pollute the Git history.

To prevent this, the `GitHookInstaller` provides a `.gitignore` template. This template ensures that only the intended artifacts are tracked by Git. Table 3.4 summarizes the versioning policy for each category of file found under the `.vitruvius/` path.

Path	Versioned	Reason
<code>.vitruvius/vsum/<branch>/</code>	Yes	Per-branch vsUM snapshot
<code>.vitruvius/changelogs/<branch>/</code>	Yes	Semantic changelog artefacts
<code>.vitruvius/branches/</code>	Yes	Branch metadata
<code>.vitruvius/versions/</code>	Yes	Version metadata records
<code>.vitruvius/merges/</code>	Yes	Persistent merge metadata records
<code>.vitruvius/*-trigger</code>	No	Transient hook signals
<code>.vitruvius/validation-result-*</code>	No	Transient validation results
<code>.vitruvius/merge-result-*</code>	No	Transient merge result files
<code>.vitruvius/*.lock</code>	No	Transient concurrency locks
<code>.vitruvius/.amend-in-progress</code>	No	Transient amendment guard*

Table 3.4: Versioning policy for `.vitruvius/` artefacts.

3.5.5.3 Consistency Guarantee

The amendment strategy provides a strong consistency guarantee. Every commit in the repository that contains model file changes also contains the corresponding vsUM snapshot. This includes both UUID mappings and the correspondence model. A developer inspecting any commit in the history can reconstruct the full vsUM state for that specific point in time.

This can be achieved without consulting any external state or performing a change replay. This property is a prerequisite for the three-way merge mechanism described in Section 3.9.1. That mechanism requires access to the common ancestor vsUM state when reconciling two divergent branches.

The guarantee holds as long as the post-commit hook is installed and the Java process is running throughout the development session, so that vsUM files are kept current on disk before each commit is made. If the hook is missing or the vsUM directory does not exist, the amendment is silently skipped. In this case, the commit will contain model files without a vsUM snapshot. Such commits are detectable by the absence of the `.vitruvius/vsum/<branchName>/` directory in the commit tree. A developer can remediate these instances by manually staging and amending the missing files.

3.6 Branch Context Switching

Branch context switching is the mechanism by which the in-memory vsUM is synchronized with the model state of a newly checked-out Git branch. The process is initiated by the post-checkout Git hook and completed by the `VsumReloadWatcher`, which invokes `BranchAwareVirtualModel.switchBranch()` as described in Section 3.5.3. This section describes the trigger file protocol that connects the Git event to the vsUM reinitialization, the locking mechanism that prevents concurrent switches, and the race condition that arises from the asynchronous communication channel.

3.6.1 Trigger File Protocol

When a developer executes `git checkout`, Git invokes the post-checkout hook with three arguments: the previous HEAD SHA, the new HEAD SHA, and a flag that distinguishes branch switches from file-level checkouts. The hook exits immediately for file checkouts and, for branch switches, writes a pipe-delimited trigger file to `.vitruvius/reload-trigger`. The file format is shown in Table 3.5. Writing the trigger and exiting immediately returns control to the Git command line without waiting for the vsUM reinitialization to complete.

Table 3.5: Structure and field definitions of the reload trigger file.

Position	Field	Description
1	<i>newBranch</i>	The name of the branch being switched to.
2	<i>requestId</i>	A unique identifier used to correlate the event in log output.
3	<i>timestamp</i>	Unix epoch time of the trigger event.
4	<i>oldBranch</i>	The name of the branch being switched from.
Separator: The pipe character (<code> </code>) is used as the field delimiter.		

We deliberately embed both branch names in the trigger file rather than resolving them from the repository at read time. By the time the `VsumReloadWatcher` detects the file, the Git HEAD has already moved to the new branch, making the previous branch name volatile and not directly recoverable without consulting the reflog. Embedding both identifiers ensures that `switchBranch()` always receives the correct `oldBranch` argument, which is critical for the `inheritFromBranchIfEmpty()` call that initializes the vsUM storage of a newly created branch from the correct parent.

3.6.2 VsumReloadWatcher

The `VsumReloadWatcher` is a background daemon thread that polls the filesystem for the reload trigger file every 500 milliseconds. Its sole responsibilities are trigger detection and thread lifecycle management. All branch-switching logic is delegated to `BranchAwareV`

`virtualModel.switchBranch()`, keeping the watcher decoupled from vsUM internals and independently testable.

When a trigger file is detected, the watcher reads and atomically deletes it, then attempts to acquire a filesystem lock on `.vitruvius/.reload.lock` using the Java `FileChannel.tryLock()` API. This non-blocking call returns immediately: if another reinitialization is already in progress the lock is unavailable, and the watcher re-creates the trigger file so the request is queued for the next polling cycle rather than silently dropped. If the lock is acquired, the watcher calls `switchBranch(oldBranch, newBranch)` and releases the lock in a `finally` block. The lock file is deleted after each release rather than retained, since `FileChannel.open()` with `CREATE` recreates it on the next acquisition attempt. The request identifier from the trigger file is included in all log output, providing a trace that links the Git event to the corresponding vsUM operation.

3.6.3 Race Condition Avoidance

A timing gap exists between the moment the post-checkout hook writes the trigger file and the moment the `VsumReloadWatcher` detects and processes it. During this window, which is bounded by the 500-millisecond polling interval, the Git repository HEAD has already moved to the new branch while the in-memory vsUM still reflects the previous one. Any model access in this interval would return stale data. This latency is bounded and inconsequential in practice, since model access rarely occurs in the milliseconds immediately after a branch switch, and the bounded delay is a known trade-off of the file-based signalling design described in Section 3.4.

A second, more subtle issue concerns branch name resolution inside `switchBranch()`. The default `VsumFileSystemLayout` constructor resolves the current branch by querying the Git HEAD via JGit. If this constructor were used inside `switchBranch()`, it would almost certainly return the new branch name, since HEAD has already moved by the time the watcher fires. While this would produce the correct target layout in most cases, it introduces a dependency on the global repository state that could conflict with concurrent Git operations. We eliminate this dependency entirely by using `VsumFileSystemLayout.forBranch(repoRoot, branchName)`, which constructs the layout directly from the branch name embedded in the trigger file. The branch switch is therefore deterministic with respect to the trigger file content and independent of any concurrent changes to the repository state.

3.7 Model Consistency Validation

The infrastructure validates the in-memory vsUM state before a Git commit is finalized, detecting structural inconsistencies that would make the committed model files unreliable or impossible to restore. Validation is initiated by the pre-commit hook and executed by

the `VsumValidationWatcher` in cooperation with the `PreCommitHandler`. The outcome determines whether the commit proceeds, proceeds with a visible warning, or is blocked.

3.7.1 Validation Flow

When a developer initiates a commit, the pre-commit hook generates a unique request identifier and writes a trigger file to `.vitruvius/validate-trigger` containing the candidate commit SHA, the branch name, the request identifier, and a Unix epoch timestamp. The hook then polls for a per-request result file named `validation-result-`. The `VsumValidationWatcher` detects the trigger within one polling cycle and attempts to acquire a `.validation.lock` via `FileChannel.tryLock()`. If the lock is unavailable because another validation is already running, the watcher re-creates the trigger file for retry on the next cycle and also writes an error result file immediately, so the hook always receives a response rather than timing out. If the lock is acquired, the watcher delegates to `PreCommitHandler.validate()` and writes the result file on completion. When validation passes, the watcher additionally updates the `lastModified` timestamp in the branch metadata file before releasing the lock. The lock file is deleted after each release. The pre-commit hook reads the result file, deletes it, and exits with status zero if validation passed or passed with warnings, or with a non-zero status code if any errors were found. The per-request identifier ensures that concurrent commit attempts from separate terminals each poll for their own result file and cannot misattribute each other's results.

3.7.2 Severity Model

`PreCommitHandler.validate()` collects issues into separate error and warning lists and constructs a `ValidationResult` from them. We distinguish three outcomes. When no issues are found the result is a clean success and the commit proceeds without any additional output. When one or more warnings are present but no errors, the result is a success with warnings: the commit proceeds and the warning messages are displayed to the developer via the hook output. When one or more errors are present the result is a failure regardless of whether warnings also exist, and the hook exits with a non-zero status code that causes Git to abort the commit. This distinction reflects the difference between structural inconsistencies that make the model state unsafe to commit and advisory conditions that are worth reporting but do not invalidate the model. If an unexpected exception is thrown during any validation step, it is caught, logged, and added to the error list, so the caller always receives a well-formed `ValidationResult` and the hook never hangs waiting for a result that will not arrive.

3.7.3 Validation Checks

We perform four checks in sequence. A later check is not skipped when an earlier one finds issues, so the developer receives a complete picture of all detected problems in a single commit attempt.

Resource loadability. The handler iterates over all model resources registered in the vsuM. Resources that are not yet in memory are loaded eagerly so that parse errors are detected immediately rather than deferred. XMI parse errors reported by EMF are added to the error list. Parse warnings are added to the warning list. If the vsuM contains no resources at all, the check adds a warning rather than an error, since an empty model may be a valid initial state.

Proxy resolution. The handler traverses all objects in every loaded resource using a recursive content iterator. Any object whose `eIsProxy()` flag is set is reported as an error. Additionally, every non-containment reference on each object is inspected by calling `eGet(ref, false)`, where the `false` argument suppresses automatic proxy resolution. References that still point to a proxy after this non-resolving read are also reported as errors. Both categories indicate that the vsuM references model content that is no longer present or has moved.

Correspondence model accessibility. The handler retrieves the correspondence model from the vsuM and verifies that the returned reference is non-null. A null value indicates a corrupted or uninitialized correspondence file and is reported as an error. Deeper validation of individual correspondence links is identified as future work, pending an enumeration API in the correspondence model view.

UUID resolver accessibility. The handler retrieves the UUID resolver and verifies that it is non-null. A null resolver indicates an uninitialized vsuM state and is reported as an error. Orphaned UUID detection, which would verify that every registered UUID maps to a currently loaded object, is similarly identified as future work.

3.7.4 Fail-Closed Policy

If no result file appears within the 30-second polling timeout, the pre-commit hook exits with a non-zero status code and blocks the commit with the message *Commit blocked for safety*. This fail-closed behavior is deliberate: allowing an unvalidated commit to proceed silently would defeat the purpose of the consistency check, since any structural inconsistency present at commit time would be permanently recorded in Git history without any indication that validation was skipped. The consequence of this policy is that the Java application must be running for Git commits on model files to proceed. During an active model development session the application is expected to be running, since it is also required for all model operations through the REST API. Commits made while the application is inactive, such as commits of non-model files, are unaffected because the post-commit hook exits early when no model files are present in the commit, as described in Section 3.4.

3.8 Semantic Change Recording

Conventional version control systems track model changes as textual diffs, which are inadequate for capturing the semantic intent behind EMF model edits. A single developer action such as renaming a classifier may produce dozens of changed lines across multiple XMI files, none of which convey the structural operation that caused them. The semantic change recording subsystem addresses this by intercepting every atomic EChange event produced by the Vitruvius propagator and persisting them at commit time as a structured JSON changelog keyed by stable element UUIDs. These records serve a dual role: they constitute a human-readable audit trail of model evolution over a branch history, and they provide the UUID-keyed input required by the conflict detection mechanism described in Section 3.9. Because the subsystem integrates through the `ChangePropagationListener` interface, it operates transparently across both the REST API commit path and the direct Git commit path, producing identical artifacts regardless of which path the developer uses. We use Vitruvius UUIDs as the primary key rather than element names or XMI paths because UUIDs remain stable across refactoring operations such as renames and relocations: two branches that independently rename the same element would otherwise appear to modify distinct elements, causing the conflict to be silently missed.

3.8.1 Architecture

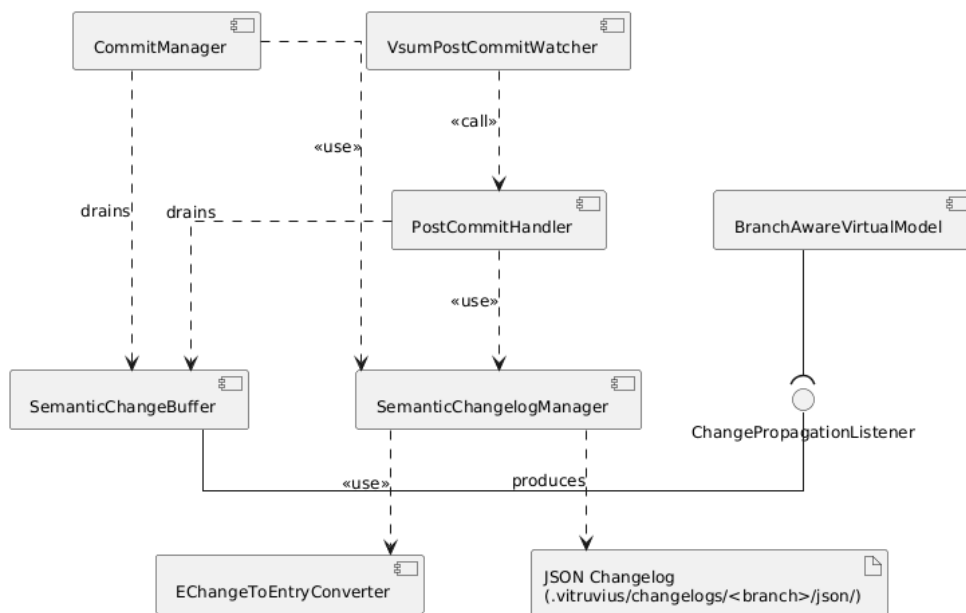


Figure 3.4: Component relationships in the semantic changelog subsystem.

Integration into the Vitruvius runtime is achieved through `BranchAwareVirtualModel`, which registers a `SemanticChangeBuffer` as a `ChangePropagationListener` on the inner

virtual model at construction time. Changes accumulate in this buffer between commits. When a commit is triggered through either path, the buffer is drained and its contents forwarded to the `SemanticChangelogManager`, which delegates to the `EChangeToEntryConverter` to produce the JSON changelog entries.

We intercept changes through the `finishedChangePropagation()` callback rather than at an earlier point in the pipeline. This callback fires only after the full propagation chain has settled, meaning all consistency reactions have executed and their produced changes are already captured within the same `PropagatedChange` list. Intercepting earlier would yield an incomplete picture where developer-initiated changes and reaction-produced changes are not yet distinguishable. The callback provides a complete and causally settled view of the change graph, which is a necessary precondition for the reaction filtering mechanism described in Section 3.8.3. The JSON changelogs produced by `SemanticChangelogManager` are subsequently read by the `SemanticConflictDetector` at merge time to compare the change histories of two divergent branches, as described in Section 3.8.6.

3.8.2 Two Commit Paths with Identical Output

A developer may commit model changes either through the REST API or directly through Git. Both paths must produce identical changelog artifacts. This requirement motivates the two-path design described below.

API path. When a client calls `CommitManager.commit()`, the system stages and commits the model files first to obtain the commit SHA. The changelog is then written synchronously using the short SHA as the filename and staged in the Git index via JGit. The next commit from the developer will therefore include the changelog file without any manual staging step. This means the changelog always resides in the commit immediately following the model changes it describes, which is a deliberate structural consequence of naming the artifact after the commit it documents.

Direct Git path. When a developer commits from a terminal or IDE, the post-commit hook writes a trigger file and exits immediately. A background daemon, the `VsumPostCommitWatcher`, detects the trigger and delegates to the `PostCommitHandler`, which drains the buffer, resolves commit metadata via JGit, and writes the changelog asynchronously. The accepted trade-off is that the changelog may not be persisted if the application terminates within the polling window following a commit.

Exactly-once semantics. Both paths share a single `SemanticChangeBuffer` instance. Whichever path executes first atomically drains the buffer via `drainChanges()`, which reads the accumulated changes and clears the buffer in a single synchronised operation. The second path then observes an empty buffer and skips changelog generation. This guarantees that each commit produces exactly one changelog entry without requiring an explicit coordination mechanism between the two paths.

```

1 BranchAwareVirtualModel model =
2     new BranchAwareVirtualModel(repoRoot, innerModel, branchManager);
3
4 // Both paths receive the same SemanticChangeBuffer reference.
5 // Whichever fires first drains the buffer; the other finds it empty.
6 CommitManager commitManager = new CommitManager(repoRoot);
7 commitManager.attachSemanticChangeTracking(
8     model.getChangeBuffer(),
9     model::getUuidResolver,
10    model::getViewSourceModels);
11
12 VsumPostCommitWatcher postCommitWatcher = new VsumPostCommitWatcher(repoRoot);
13 postCommitWatcher.attachSemanticChangeTracking(
14     model.getChangeBuffer(),
15     model::getUuidResolver,
16     model::getViewSourceModels);

```

Listing 3.2: Both commit paths wired to the shared SemanticChangeBuffer.

3.8.3 Reaction Change Filtering

In Vitruvius `TRANSITIVE_CYCLIC` propagation mode, a single developer-initiated change may trigger a cascade of consistency reactions. A UML model edit, for example, might cause the automatic derivation of corresponding Java artifacts. The propagator delivers both the original change and all reaction-produced changes to the `finishedChangePropagation()` callback within the same list of `PropagatedChange` objects. Recording all delivered changes indiscriminately would conflate explicit developer intent with automatically derived consistency maintenance, which would pollute the changelog and interfere with conflict detection.

We filter reaction-produced changes using a behavioural invariant of the Vitruvius propagator: the `EChange` instances recorded in the `consequentialChanges` field of a `PropagatedChange` are reused by reference as the `originalChange` of the reaction's own `PropagatedChange`, without defensive copying. Java object-identity comparison is therefore sufficient to identify and exclude them. We collect all consequential `EChange` instances into an identity-keyed set and then skip any `PropagatedChange` whose original changes are entirely contained in that set.

```

1 List<PropagatedChange> pcList = new ArrayList<>();
2 propagatedChanges.forEach(pcList::add);
3
4 // Build an identity set of all EChange instances that appear as
5 // consequential (reaction) changes across the full propagation list.
6 Set<Object> reactionEChanges =
7     Collections.newSetFromMap(new IdentityHashMap<>());

```

```
8  for (PropagatedChange pc : pcList) {
9      VitruviusChange<EObject> consequential = pc.getConsequentialChanges();
10     if (consequential != null) {
11         reactionEChanges.addAll(consequential.getEChanges());
12     }
13 }
14
15 // Skip any PropagatedChange whose original EChanges are entirely
16 // contained in the reaction set; all others are user-initiated.
17 for (PropagatedChange pc : pcList) {
18     VitruviusChange<EObject> original = pc.getOriginalChange();
19     List<EChange<EObject>> eChanges = original.getEChanges();
20     if (!eChanges.isEmpty() && reactionEChanges.containsAll(eChanges)) {
21         continue;
22     }
23     collectFromVitruviusChange(original);
24 }
```

Listing 3.3: Reaction change filtering in `SemanticChangeBuffer.finishedChangePropagation()`.

This mechanism relies on an internal implementation property of the propagator rather than a formally specified contract. A future refactoring of the Vitruvius propagator that introduces defensive copying of `EChange` instances would silently invalidate the filter, causing reaction changes to appear in the changelog without any compile-time or runtime indication of failure. We document this assumption explicitly as the primary fragility point of the subsystem.

3.8.4 Change Classification

Each atomic change is assigned to one of fifteen categories defined by the `SemanticChange` Type enumeration, listed in Table 3.6. The classification maps directly onto the EMF atomic change hierarchy and is designed to satisfy two goals: interpretability for developers who are unfamiliar with the EMF change model, and sufficient precision for automated conflict classification during branch integration. Each constant carries a human-readable description string that is embedded in the JSON changelog, making the record self-explanatory without additional tooling.

Constant	EMF Type	Description
ELEMENT_CREATED	CreateEObject	A model element was instantiated
ELEMENT_DELETED	DeleteEObject	A model element was destroyed
ROOT_INSERTED	InsertRootEObject	A root element was added to a resource
ROOT_REMOVED	RemoveRootEObject	A root element was removed from a resource
ATTRIBUTE_SET	ReplaceSingleValuedEAttribute	Attribute received its first value
ATTRIBUTE_CHANGED	ReplaceSingleValuedEAttribute	Attribute value was replaced
ATTRIBUTE_CLEARED	ReplaceSingleValuedEAttribute	Attribute was reset to default
ATTRIBUTE_VALUE_INSERTED	InsertEAttributeValue	Value inserted into attribute list
ATTRIBUTE_VALUE_REMOVED	RemoveEAttributeValue	Value removed from attribute list
REFERENCE_SET	ReplaceSingleValuedEReference	Reference linked for the first time
REFERENCE_CHANGED	ReplaceSingleValuedEReference	Reference target replaced
REFERENCE_CLEARED	ReplaceSingleValuedEReference	Reference unlinked
REFERENCE_VALUE_INSERTED	InsertEReference	Element added to reference list
REFERENCE_VALUE_REMOVED	RemoveEReference	Element removed from reference list
ELEMENT_REORDERED	PermuteContainmentEReference	Elements rearranged in containment

Table 3.6: SemanticChangeType constants and their corresponding EMF types.

The three-way subdivision of single-valued attribute and reference changes into SET, CHANGED, and CLEARED is a deliberate design decision. This granularity enables fine-grained conflict classification during merge. A write-write conflict of elevated severity arises when two branches independently apply a CHANGED operation to the same structural feature of the same element. A lower-severity divergence occurs when one branch issues a SET operation on a previously unassigned feature while the other branch leaves it unmodified, since the intent of the two operations is not directly contradictory.

3.8.5 Changelog Format

The changelog artifact is a JSON file named `.json`, stored under `.vitruvius/changelogs/<branchname>/json/`. Naming the file after the abbreviated commit SHA establishes a direct correspondence between changelog entries and `git log` output, enabling traceability without auxiliary lookup structures. The document structure contains four top-level sections: a format version tag for schema evolution, a commit metadata block, an ordered list of per-file change records, and a summary. The listing below shows an abbreviated changelog produced when the developer on the `feature-diameter` branch in Scenario S1 increases `disk1.diameterInMM` from 120 to 130.

```

1 {
2   "formatVersion": "1.0",
3   "commit": {
4     "sha": "a3f8c92d1e4b7f60d8e29a1c3b5d7f9e2c4a6b8d",
5     "shortSha": "a3f8c92",
6     "branch": "feature/resize-disk",
7     "author": { "name": "Developer A", "date": "2026-05-26T14:30:00" },
8     "message": "Increase brake disk diameter",
9     "parentShas": [ "9b2e7a4c6f1d3e8b2a0c4d6e8f1a2b3c4d5e6f7a" ]

```

```
10  },
11  "fileChanges": [
12    {
13      "operation": "MODIFIED",
14      "path": "model/brakesystem.xmi",
15      "oldPath": null,
16      "semanticChanges": [
17        {
18          "index": 0,
19          "changeType": "ATTRIBUTE_CHANGED",
20          "elementUuid": "uuid-disk1-a1b2c3",
21          "containerUuid": "uuid-root-d4e5f6",
22          "eClass": "brakesystem::BrakeDisk",
23          "feature": "diameterInMM",
24          "from": "320",
25          "to": "340"
26        }
27      ]
28    }
29  ],
30  "summary": {
31    "totalFileChanges": 1,
32    "totalSemanticChanges": 1,
33    "affectedElementUuids": [ "uuid-disk1-a1b2c3" ]
34  }
35 }
```

Listing 3.4: Abbreviated JSON changelog entry for a single commit.

Each file-level record carries an operation field classifying the change as ADDED, MODIFIED, DELETED, or RENAMED, derived from the Git diff against the commit's first parent. Each semantic change entry within a file record includes the containerUuid field recording the Vitruvius UUID of the immediate EMF container of the affected element. This field enables a future tombstoning mechanism that can detect conflicts where a parent element is deleted on one branch while a child of that parent is modified on the other. The parentShas field in the commit block supports three-way merge algorithms by providing a direct reference to the common ancestor. The summary.affectedElementUuids list serves as a fast pre-filter for conflict detection: two commits that share no element UUIDs in their summaries cannot conflict and require no further comparison.

The choice of Vitruvius UUIDs as the primary conflict detection key is significant in a branching context. In a single-branch history, element names or XMI paths might serve as adequate surrogates for identity. In a multi-branch scenario, however, two branches may independently rename the same element, and a name-based key would cause the conflict detector to treat them as changes to two distinct elements, silently missing the conflict. UUID stability across refactoring operations is therefore a correctness requirement rather than a convenience.

3.8.6 Change Replay and Conflict Detection

The `SemanticConflictDetector` implements a multi-step pipeline for detecting and classifying conflicts between two divergent branches. Rather than comparing model files directly, the detector operates on the JSON changelogs produced by the `SemanticChangelogManager`, making conflict detection independent of file position and serialisation order. The pipeline covers four phases: locating the common ancestor and loading changelog records, direct conflict detection with origin-based auto-resolution, dependency graph construction and topological ordering, and change replay. The first two phases are a contribution of this thesis and execute within the `SemanticConflictDetector`. Phases three and four are provided by the `SemanticMergeEngine` developed in the accompanying research work [23], and are integrated into the pipeline as described in Section 3.9.

Common Ancestor and Changelog Loading. We first locate the common ancestor of the two branches using JGit’s merge-base algorithm, which identifies the most recent commit shared by both histories. Only commits made after this point differ between the branches and are therefore relevant for conflict detection. We then collect the short SHAs unique to each branch since the ancestor and use them to locate the corresponding JSON changelog files. These files are read from the Git object store by walking the HEAD commit tree of each branch, so no branch checkout is required. Changes across multiple model files within a single commit are flattened into one list of `SemanticChangeEntry` records for uniform comparison in the next phase.

Direct Conflict Detection. The detector compares the two changelog entry sets pairwise over the key (`elementUuid`, `feature`). No model or resource set is loaded during this step: all comparisons operate on the UUID strings and feature names serialised into the changelog JSON. Two entries on the same key conflict only when they record different target values. When both entries record the same target value, the change is auto-resolvable and no conflict record is created. For list-valued features, the comparison is finer-grained. Two concurrent INSERT operations targeting different element UUIDs on the same feature are treated as independent and produce no conflict, because both insertions can be applied to the list without contradiction. An INSERT on one side and a REMOVE targeting the same element UUID on the other produces a HIGH-severity conflict, since the two operations directly contradict each other. When one side records a deletion of an element and the other records a modification of the same element, the conflict is classified as HIGH, because the deleted element is no longer a valid modification target. When both sides delete the same element, no conflict is recorded, since the outcome is identical regardless of which change is applied. For non-list reference features, conflicting entries are classified as HIGH. Conflicting entries on scalar attribute features are classified as MEDIUM. When more than one conflict is found for the same (`elementUuid`, `feature`) key, only the highest-severity record is retained.

Tombstone Conflict Detection. A second category of conflict arises when one branch deletes a model element while the other branch modifies a child of that element. We call this a

tombstone conflict, because the deletion leaves any surviving child changes orphaned. To detect these cases, we construct a deletion map for each branch from all `ELEMENT_DELETED` entries. The deletion map is populated with the UUID of the directly deleted element and with all UUIDs listed in its `cascadeDeletedUuids` field, which records the descendants that were removed transitively at changelog capture time. Two checks then run bidirectionally across the two maps. The first check is the container tombstone check: for each change entry on one branch, we test whether its `containerUuid` field, which records the UUID of the element's immediate EMF container, appears in the deletion map of the other branch. A match means the change targets an element whose parent was deleted, making the child an orphan, and we record a `HIGH`-severity conflict. The second check is the cascade cross-check: we compare the full deletion map of each branch against the set of all modified element UUIDs on the other branch. This catches cases where a deeply nested descendant of a deleted element was also modified on the opposing branch, which the container tombstone check alone cannot detect because it only inspects the immediate parent. Both checks share the same deletion map, so cascade-deleted descendants are covered by both.

Origin-Based Auto-Resolution. After direct and tombstone conflicts have been identified, we apply an automatic resolution pass over the collected conflict list. Each changelog entry carries a `changeOrigin` field with one of three values: `ORIGINAL`, `CONSEQUENTIAL`, or `UNKNOWN`. An `ORIGINAL` change is one the developer issued directly, such as setting an attribute or adding a reference. A `CONSEQUENTIAL` change is a side effect recorded by the Vitruvius reaction machinery in response to an original change, such as a complementary update applied to a mirrored model element to maintain consistency. An `UNKNOWN` origin is assigned to entries recorded before origin tracking was introduced, serving as a backward-compatibility sentinel for older changelog files. When a conflict pairs an `ORIGINAL` entry from one branch against a `CONSEQUENTIAL` entry from the other, we resolve it automatically: the `ORIGINAL` intent takes precedence and the `CONSEQUENTIAL` side effect is discarded, since it was generated in reaction to a deliberate developer decision. When both conflicting entries are classified as `ORIGINAL`, or when either entry carries an `UNKNOWN` origin, the conflict cannot be resolved automatically, because no reliable precedence rule can be applied. If any conflicts remain unresolved after this pass, the `SemanticConflictDetector` returns them to the caller immediately and phases three and four are not entered within the detector itself. When both a `ConflictResolutionProvider` and a `SemanticMergeEngine` are configured and direct conflicts remain, the `MergeManager` creates a provider-aware engine instance via `mergeEngine.withConflictResolutionProvider()` and invokes the directed `merge()` method, which re-detects conflicts internally via `UuidConflictDetector` and applies the provider per conflicting element to filter the source branch change sequence before replay. This path is described in Section 3.9. In practice the auto-resolution pass reduces the conflict set substantially for projects that use Vitruvius consistency rules, since each original change typically generates multiple consequential updates that would otherwise surface as spurious conflicts.

Dependency Graph and Replay. When no conflicts remain after auto-resolution, the pipeline proceeds to phases three and four, which are delegated to the `SemanticMergeEngine` when one is wired into the detector. The engine receives the full commit SHAs of both

branch heads and the common ancestor and executes a bidirectional replay strategy via `mergeBidirectional()`: it first attempts a forward replay, routing source branch changes through the target vsUM so that consistency preservation rules fire. If that direction produces indirect conflicts, it attempts the reverse direction. If both directions yield indirect conflicts, the engine escalates to a bidirectional indirect conflict that requires human resolution. Replay is the primary merge step rather than a fallback: routing source branch changes through the live vsUM via `propagateChange()` ensures that consistency preservation rules fire and propagate their consequential updates, which a textual Git merge of XMI files would not trigger. Additionally, replay can expose indirect conflicts that the changelog pre-check cannot foresee, because a CPR triggered by a source branch change may overwrite an element the target branch already modified directly, a contradiction that only surfaces when the reactions execute. The engine also provides a dependency-graph-based interleaving strategy via `mergeWithInterleaving()`: commits from both branches are arranged in a footprint dependency graph, cycles in that graph indicate interleaving conflicts, and a topological sort of the acyclic graph yields the replay order. A change depends on another when it targets an element that the other change creates or relocates, since the creation must be applied before the dependent change can address a valid target, as illustrated in Figure 3.5. In the figure, Change A represents an element creation event, such as adding `sensor1` to the Brakesystem in Scenario S1, and Change B is an attribute modification that targets the newly created element. Because Change B requires `sensor1` to exist, the topological sort guarantees that Change A is replayed first. When no merge engine is provided, the detector falls back to a stub pipeline: `buildDependencyGraph()` is called but currently returns an empty map, because the consequential footprint data required to populate it is not yet exposed by the change tracking layer. The subsequent topological sort and replay steps then operate on an unordered change set. Populating the dependency graph fully from changelog data is identified as future work, contingent on extending the changelog entry format to carry the set of element UUIDs that each change indirectly affects through the reaction machinery.

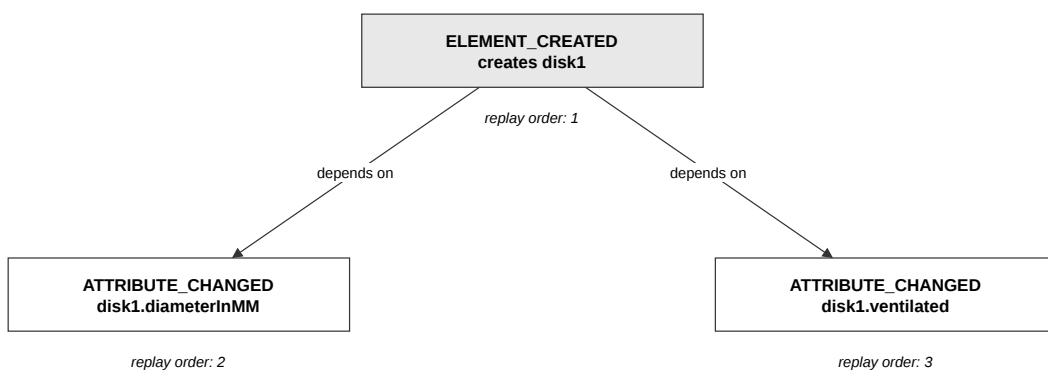


Figure 3.5: Dependency graph example: Change B depends on Change A because A creates the element that B modifies. Topological replay executes A before B.

3.9 Merge Handling

Merging within a branch-aware vsUM system is fundamentally more complex than branch switching. Whereas a branch switch replaces the in-memory model state with a pre-existing per-branch snapshot, a merge must produce a new consistent state from two divergent histories, each of which may carry UUID mappings, correspondence model entries, and semantic changelog records that are mutually inconsistent. The central challenge specific to Vitruvius is that inter-model consistency is maintained by reaction rules that fire during change propagation, not by static file content. A purely file-level merge of the XMI serialisations would produce syntactically valid files while leaving the models semantically inconsistent, because the reactions would not have fired for the merged content. We address this through a multi-stage pipeline coordinated by the MergeManager component, which routes each merge through a semantic pre-check, followed by either engine-based change replay or a JGit fallback depending on the configuration. Two entry points converge on this pipeline: the REST API invokes `MergeManager.merge()` directly, while the Git command-line path registers a `GitMergeDriver` in `.gitattributes` that intercepts file-level conflicts and delegates to the same `SemanticMergeEngine`.

3.9.1 Merge Scenario Classification

We classify merges into three categories based on the ancestry relationship between the two branch heads and the outcome of the semantic pre-check. The category determines which stage of the pipeline is activated and what post-merge actions are performed.

Fast-Forward Merge. A fast-forward situation arises when the two branch heads are in a direct ancestry relationship. We check this using JGit's `isMergedInto()` predicate in both directions before touching any file content. If the source head is already an ancestor of the target head, the target branch already contains all source branch changes and no merge action is needed. If the target head is an ancestor of the source head, Git can advance the target branch pointer to the source head without creating a merge commit, since the source constitutes a strict extension of the target history. In this case we invoke JGit with the fast-forward-only strategy, then copy the source branch vsUM directory into the target, reload the in-memory model, and mark the source branch as MERGED in its metadata. We also copy the source branch changelog JSON files into the target branch changelog directory so that the merged branch's full commit history remains accessible through the REST API. No engine replay step is required in either fast-forward case.

Non-Fast-Forward Merge Without Semantic Conflicts. When neither branch is an ancestor of the other, we first run the semantic pre-check using a lightweight `SemanticConflictDetector` instance covering steps one through three of the pipeline described in Section 3.8.6. The pre-check compares the recorded changelogs of both branches and returns a list of conflict descriptors of the form `semantic://{uuid}/{feature}` for each detected conflict, using `lifecycle` as the feature name for element creation and deletion conflicts. If the list is

empty, no element-level contradiction exists. When a `SemanticMergeEngine` is configured, we bypass JGit file-level merging entirely and proceed directly to engine replay. We set `MERGE_HEAD` manually to the source branch commit so the resulting commit carries two parents, then invoke `SemanticMergeEngine.merge()` with the common ancestor SHA and both branch head SHAs. The engine replays the source branch changes onto the target vsum state, writes the merged model files to a temporary folder, and returns a result object. We copy those files to the working tree, stage them, and create the merge commit. Bypassing JGit is deliberate: a textual merge of XMI files can produce conflict markers that make the files unparseable by EMF, even when the underlying model changes do not logically conflict. When no engine is configured, the pipeline falls back to a JGit three-way merge using the `RECURSIVE` strategy.

Non-Fast-Forward Merge With Semantic Conflicts. When the semantic pre-check returns a non-empty conflict list, the pipeline branches on whether a `ConflictResolutionProvider` has been registered. If no provider is set, the merge is blocked immediately and the conflict descriptor list is returned to the caller. No file-level operation is performed and the repository is left in its pre-merge state, allowing the developer to inspect the conflicts and decide how to proceed. When a provider is set and a `SemanticMergeEngine` is also configured, we invoke `resolveWithEngine()`, which delegates to the engine's replay pipeline with the provider consulted per conflicting element. This is a per-element resolution: non-conflicting changes from both branches are preserved as-is, while conflicting elements are resolved individually according to the provider's `THEIRS` or `OURS` choice. The result is a two-parent merge commit that incorporates both branches' non-conflicting changes and applies the developer-specified resolution for each contested element. When a provider is set but no engine is available, the pipeline falls back to a coarse file-level JGit merge using the strategy inferred from the provider, which applies a single `THEIRS` or `OURS` decision uniformly across all conflicting model files.

3.9.2 Implemented Merge Pipeline

The `MergeManager.merge()` method coordinates the full pipeline. It opens the repository, resolves the source branch reference, detects fast-forward cases before touching any file, runs the semantic pre-check for non-fast-forward cases, and then routes to one of three execution paths depending on the engine and provider configuration: engine replay, per-element conflict resolution, or JGit file-level merge.

Fast-Forward Detection. Before invoking any merge operation, we open a `RevWalk` and use `isMergedInto()` to check the ancestry relationship between the two branch heads. This check is purely a commit-graph traversal and does not modify any file content. We probe both directions. If the source head is already reachable from the target head, the target branch already incorporates all source branch changes and we return a fast-forward result immediately without further action. If the target head is reachable from the source head, a true fast-forward is possible: we invoke JGit with `FastForwardMode.FF_ONLY` to advance the branch pointer, then copy the source branch vsum directory into the target, copy the source

branch changelog files into the target branch changelog directory, reload the in-memory model, and mark the source branch as MERGED. The changelog copy ensures that after the merge, queries for the target branch changelog history include all commits made on the source branch.

```

1 SOURCE_BRANCH=$(git name-rev --name-only HEAD@{1} 2>/dev/null \
2   | sed 's|^remotes/origin/||' \
3   | sed 's/~[0-9]*$//')
4 SOURCE_VSUM="$REPO_ROOT/.vitruvius/vsum/$SOURCE_BRANCH"
5 TARGET_VSUM="$REPO_ROOT/.vitruvius/vsum/$TARGET_BRANCH"
6
7 if [ -d "$SOURCE_VSUM" ] && [ "$SOURCE_BRANCH" != "$TARGET_BRANCH" ]; then
8   rm -rf "$TARGET_VSUM"
9   cp -r "$SOURCE_VSUM" "$TARGET_VSUM"
10 fi

```

Listing 3.5: Fast-forward vsum copy in the post-merge hook (CLI path).

Semantic Pre-Check. Before any file-level merge is attempted, we run a semantic pre-check using the `SemanticConflictDetector`. This component compares the recorded changelogs of both branches and identifies changes that target the same model element and structural feature. Each detected conflict is encoded as a descriptor of the form `semantic://uuid/feature`, where `uuid` identifies the conflicting model element and `feature` names the structural feature on which both branches recorded a modification. For element creation or deletion conflicts, the feature is set to `lifecycle`. Listing 3.6 shows the descriptors returned when both branches modify `disk1.diameterInMM` as in Scenario S2 of our running example.

```

1 semantic://uuid-disk1-a1b2c3/diameterInMM
2 semantic://uuid-disk1-a1b2c3/lifecycle

```

Listing 3.6: Example conflict descriptors returned by the semantic pre-check.

If the pre-check finds no conflicts, the pipeline routes to engine replay when a `SemanticMergeEngine` is configured, or falls through to the JGit merge when none is configured. If conflicts are found and no resolution strategy has been provided, the merge is blocked and the descriptors are returned to the caller. If a resolution strategy is available, the pipeline skips both engine replay and the JGit merge and proceeds directly to conflict resolution, described below.

Engine Replay When no semantic conflicts are found and a `SemanticMergeEngine` is configured, engine replay is the primary merge path. The motivation for this design is that a textual merge of XMI model files would leave the vsum semantically inconsistent: Vitruvius inter-model consistency reactions fire during change propagation, not on static file content, so a file-level merge bypasses them entirely. By replaying the original EChange records through the live Vitruvius reaction engine, we ensure that all consistency reactions fire during the merge itself and the merged state is immediately valid without a separate reconciliation step. We first write `MERGE_HEAD` manually to the source branch commit,

causing Git to treat the subsequent commit as a two-parent merge commit. We then resolve the common ancestor SHA using a `RevWalk` with the `MERGE_BASE` filter and invoke `SemanticMergeEngine.merge(baseSha, oursSha, theirsSha)`. This call is the execution counterpart to the `mergeBidirectional()` call made during the semantic pre-check: the pre-check verified the absence of conflicts; this call writes the merged model state to disk. The engine deserialises the original changes from the source branch changelogs, replays them onto the target `VSUM` state, and returns a `SemanticMergeResult` containing a folder of merged model files. If the engine reports failure, we perform a hard reset and return a conflicting result. On success, we copy only the merged model files to the working tree, excluding `.vitruvius/` and `.git/` to avoid stale absolute URIs, stage them with targeted `git add` patterns to prevent unrelated artefacts entering the commit, and create the merge commit tagged with `[semantic-replay]`.

JGit Fallback. When no merge engine is configured, the pipeline falls back to a JGit three-way merge using the `RECURSIVE` strategy. This path is also taken when a `ConflictResolutionProvider` is set but no semantic conflicts were detected, which can occur when a file rename bypasses the changelog-based pre-check and the developer has supplied a resolution strategy preemptively. In both cases, JGit reconciles the model files at the textual level and creates a merge commit if all files resolve cleanly. If JGit reports a `CONFLICTING` status, the conflicting file paths are collected from the merge result and returned to the caller. No hard reset is performed in this case, since the partial merge state may be useful for manual inspection. When a `ConflictResolutionProvider` is set, the JGit strategy is inferred from the provider by probing it with a dummy conflict and checking whether the returned choice is `THEIRS` or `OURS`, applying that strategy uniformly across all model files.

Per-Element Conflict Resolution. When the semantic pre-check detects conflicts and both a `ConflictResolutionProvider` and a `SemanticMergeEngine` are configured, we invoke `resolveWithEngine()`. The method first writes `MERGE_HEAD` to the source branch commit so the result is a proper two-parent merge commit, then resolves the common ancestor SHA using the same `RevWalk` approach as the engine replay path. A new provider-aware engine instance is created by calling `mergeEngine.withConflictResolutionProvider()` with the registered provider, rather than modifying the shared engine. This instance then calls `merge(baseSha, oursSha, theirsSha)`, the directed replay variant, which is the only path that invokes `ConflictResolutionProvider.resolve()` on detected conflicts during the replay. The provider's `THEIRS` or `OURS` choice is applied per conflicting element, while non-conflicting changes from both branches are preserved as-is. If the engine cannot resolve all conflicts, we perform a hard reset and delete `MERGE_HEAD` to leave the repository in a clean state before returning a conflicting result. On success, the merged model files are copied and staged using the same mechanism as the engine replay path, and the merge commit is tagged with `[semantic-resolved]` to distinguish it from a plain replay merge.

Post-Merge Actions. After a successful merge, regardless of which path produced the merge commit, we execute four follow-up steps in sequence. First, we update the source branch metadata file to the `MERGED` state and create a separate system commit tagged

with `Vitruvius-System:true` so the status change is recorded in Git history without the developer needing to stage it manually. Second, we copy the source branch changelog JSON files from the Git object store into the target branch changelog directory and commit them in another system commit, making the merged branch's full changelog history visible when the target branch changelog is queried through the REST API. Third, in REST mode we invoke the registered `PostMergeHandler` to copy the source branch `vsUM` directory into the target branch directory, ensuring that resources introduced on the source branch are known to the target `vsUM`, and then call the `postMergeReload` callback to reload the in-memory model from the merged files. In CLI mode both of these steps are handled by the `VsumMergeWatcher`, which is triggered by the merge trigger file written during the merge rather than by a direct callback. Fourth, if the caller requested branch deletion, we delete the source branch after all other steps have completed, so a failure in any earlier step leaves the source branch intact and the merge can be retried.

3.10 Versioning

Branch-based development allows developers to isolate changes, but it does not on its own provide a mechanism for capturing and restoring specific stable states within a branch. A developer may want to mark the current model as a known-good baseline before applying a risky refactoring, or share a named reference point with a reviewer without creating a separate branch. The `VersioningService` addresses this by implementing a two-step versioning mechanism built on top of Git annotated tags. A version is a named, immutable snapshot of the model state at a specific commit. It can be used to inspect the model at that point in time, roll back the branch to that state, or create a new branch initialised from it.

When a version is created, the `VersioningService` first writes a metadata file to `.vitruvius/versions/<versionId>.metadata` and commits it to the repository. It then creates a Git annotated tag pointing to that commit. The tag therefore identifies a commit that already contains its own metadata, making the version self-describing within the Git history. The metadata record captures the version identifier, the full commit SHA, the branch on which the version was created, the author name and email, the creation timestamp, and an optional human-readable description. A version also carries a maturity level, initialized to `DRAFT`, which follows the same convention as branch maturity and serves as an extension point for future editorial workflows. Version identifiers must conform to Git tag naming rules and are validated before the tag is created.

Rollback to a named version is implemented as a two-step process to prevent accidental data loss. In the first step, the developer calls `previewRollback()`, which resolves the target commit from the Git tag and computes the impact of the rollback without modifying any files. The preview reports the list of commits that would be abandoned, the files that would change relative to the current `HEAD`, and whether any tracked files carry uncommitted modifications. The check uses a content-level diff so that model files re-saved by EMF with unchanged byte content do not produce false positives. In the second step, the developer calls `confirmRollback()` with the preview object. The service rejects the confirmation if

such modifications are detected, since a hard reset would permanently overwrite them. If the working directory is clean, the service executes `git reset -hard` to the target commit and invokes the reload callback to synchronize the in-memory vsUM with the restored model files. The result object indicates whether the reset and the subsequent reload both succeeded, since a failed reload leaves the repository in the correct state on disk but with a stale in-memory model.

The `VersioningService` also supports creating a new branch whose starting state is initialised directly from a named version rather than from the current HEAD. When `createBranchFromVersion()` is called, the service resolves the commit referenced by the version tag and creates a new Git branch pointing to that commit. It then walks the commit tree using JGit to extract the vsUM files stored under `.vitruvius/vsum/<sourceBranch>/` and writes them to `.vitruvius/vsum/<newBranch>/` on the local filesystem. This extraction step is necessary because a standard Git branch creation only moves the branch pointer without touching the working directory. Finally, branch metadata is written for the new branch with the version's source branch recorded as the parent. The result is a branch that starts from a well-defined historical model state and is immediately usable without any manual vsUM initialization.

The choice to implement versions as Git annotated tags rather than as a separate versioning database was deliberate. Annotated tags are a first-class Git construct, persistent across repository clones, and visible in standard Git tooling. This means that a version created through the `VersioningService` is immediately inspectable with `git tag` and `git show`, and is transferred automatically during `git fetch` and `git push` without any additional synchronization mechanism. The accompanying metadata file committed alongside each tag ensures that version descriptions, author information, and maturity levels remain part of the repository history even if the tag itself is later deleted.

3.11 REST API

The Vitruv-Server module wraps the vsUM runtime in an HTTP layer, allowing remote clients to perform all model operations through a stateless JSON API without requiring a direct Java dependency on the framework. The server is implemented as a `VitruvServer` class that accepts a `VirtualModelInitializer` and an optional set of manager instances at construction time. It initializes the vsUM internally, constructs a `JsonMapper` bound to the repository root, and registers the corresponding endpoint groups via the `Endpoint sProvider` builder. Endpoint groups are conditional: a group is only registered when its backing manager is supplied. This means that a deployment that omits the `MergeManager`, for example, will expose no merge endpoint, making the API surface reflect exactly the capabilities of the configured server instance. The underlying HTTP transport is provided by Java's built-in `HttpServer`, wrapped in a `VitruvJavaHttpServer` that routes incoming requests to the matching endpoint by path and HTTP method.

All endpoints follow a uniform convention. Specific resources are identified by path parameters, for example `/vsum/branch/{branchName}` or `/vsum/version/{versionId}`, which is consistent with standard REST practice. Optional filtering parameters, such as the base branch name used by the delta endpoint, are passed as query parameters. Request bodies, where present, are JSON-encoded and parsed by the `JsonMapper`. All successful responses return `application/json` with a structured JSON body. Error responses return an appropriate HTTP status code together with a plain-text message describing the cause. The underlying routing layer extracts path segments relative to the registered context prefix and makes them available to each endpoint handler, allowing the path-parameter convention to be applied uniformly across all endpoint groups without modifying the existing method-dispatch logic in `RequestHandler`.

3.11.1 Endpoint Design

Tables 3.7 and 3.8 list all endpoints introduced as part of this thesis, grouped by functional category. The base V-SUM endpoints inherited from the existing Vitruv-Server remote module[30], covering view retrieval, change propagation, and view type discovery, remain available and are not listed here since they are not part of this work.

3.11.2 Visualization and Version Browsing

A developer working with a branch-aware vsum needs two kinds of inspection capability beyond raw model access: the ability to see what semantic changes a branch introduced relative to its parent, and the ability to inspect what the model looked like at a specific named version. We address both needs through four additional endpoints that produce structured, human-readable output rather than raw model content. The design choice to serve these outputs from the REST layer rather than from a separate tool is deliberate: because the semantic changelogs and version metadata already reside in the Git object store and are accessible through the running server, a client needs only a standard HTTP tool such as `curl` to retrieve a formatted diff, a commit history, or a browser-renderable model graph. No additional process or build step is required.

Semantic delta visualization. The `GET /vsum/branch/{branchName}/history` endpoint returns the commit history of a branch with inline semantic change summaries. For each commit it includes the commit SHA, message, author timestamp, total semantic change count, and a preview of up to three changes formatted as human-readable strings of the form `ELEMENT_DELETED brakesystem::BrakeDisk.diameterInMM: 120 -> 130`. Bundling the commit list with the inline change previews avoids the $N + 1$ round trips that would arise if a client queried the commit list and then fetched each changelog separately. The `GET /vsum/delta/{branchName}` endpoint addresses the cross-commit dimension: it aggregates all semantic changes introduced on the given branch since it diverged from a configurable base branch, which defaults to `main`. The divergence point is computed using `JGit's RevFilter.MERGE_BASE`, which performs a commit-graph traversal without touching

Table 3.7: Branch, commit, and merge endpoints introduced by this thesis.

Path	Method	Description
Branch Management		
/vsum/branch	GET	Lists all known branches with metadata.
/vsum/branch	POST	Creates a new branch from a given parent.
/vsum/branch/{branchName}	GET	Returns full metadata for a single branch.
/vsum/branch/{branchName}	DELETE	Deletes a branch.
/vsum/branch/{branchName}/switch	POST	Switches the active branch.
/vsum/branch/{branchName}/state	GET	Returns the lifecycle state of a branch.
/vsum/branch/{branchName}/maturity	PATCH	Updates the maturity level of a branch.
/vsum/branch/{branchName}/history	GET	Returns commit history with inline semantic change previews.
/vsum/branch/{branchName}/conflicts	GET	Returns a preview of semantic conflicts that would arise from merging this branch.
/vsum/branch/topology	GET	Returns full parent-child branch hierarchy.
Commit and Changelog		
/vsum/commit	POST	Stages model files and creates a Git commit including semantic changelog generation.
/vsum/commit/{branchName}	GET	Lists commits on the given branch.
/vsum/changelog/{branchName}/{sha}	GET	Returns JSON changelog for a certain commit.
/vsum/delta/{branchName}	GET	Returns the aggregated semantic delta between the given branch and a base branch
Merge		
/vsum/merge	POST	Merges a source branch into the active branch with an optional resolution strategy supplied in the request body.

any file content. The response groups all changes by commit shortSha, giving the developer a flat, ordered list of every semantic operation performed on the branch since the fork. In the running example, a developer working on a feature-diameter branch that modified disk1.diameterInMM from 120 to 130 can retrieve the full delta relative to main in a single request and immediately see which element and which attribute were touched, without inspecting the raw model files.

Version inspection. The GET/vsum/version/{versionId}/model endpoint returns the model state captured at a specific named version. Given a version identifier, it resolves the corresponding Git annotated tag through the VersioningService, retrieves the commit SHA recorded in the version metadata, and reads the vsum model files directly from the Git object store at that commit using a JGit TreeWalk. Reading from the object store rather than the working directory ensures that the response always reflects the exact state captured at the version, regardless of any uncommitted changes or branch switches that may have occurred since. The endpoint supports three output formats selected via a query

Table 3.8: Versioning endpoints introduced by this thesis.

Path	Method	Description
Versioning		
/vsum/version	GET	Lists all versions on the active branch.
/vsum/version	POST	Creates a named version from the current branch state.
/vsum/version/{versionId}	GET	Returns metadata for a specific version.
/vsum/version/{versionId}	DELETE	Deletes a named version.
/vsum/version/{versionId}/rollback/preview	POST	Returns the commits that would be abandoned by a rollback.
/vsum/version/{versionId}/rollback/confirm	POST	Executes the rollback to the named version.
/vsum/version/{versionId}/branch	POST	Creates a new branch from a specific version.
/vsum/version/{versionId}/model	GET	Returns the model state at a specific version.
/vsum/version/{versionId}/view	GET	Returns a view derived from the model state at a specific version.

parameter. The default text format renders the model as an ASCII containment tree, which is suitable for inspection in a terminal. The json format returns a nested JSON structure for programmatic consumption. The mermaid format emits a Mermaid graph TD (top-down) string that can be embedded in documentation or rendered by a compatible tool. The GET /vsum/version/{versionId}/view endpoint complements this by returning a self-contained HTML page that embeds the Mermaid graph and loads the Mermaid.js renderer from a public Content Delivery Network (CDN). Opening the response body directly in a browser produces an interactive containment graph without requiring any local installation. In the running example, after tagging version v1.0 containing disk1 and sensor1 at their stable dimensions, a reviewer can retrieve the text tree of that exact model state and compare it against the current branch without checking out the tagged commit.

4 Related Work

The design of branching support for Vitruvius requires evaluation of existing model versioning approaches. This chapter examines state-of-the-art frameworks for model comparison, versioning, and conflict detection, analyzing their suitability for integration with Vitruvius’s consistency-preservation architecture. Each approach is assessed based on compatibility with Vitruvius’s operation-based change tracking, architectural overhead, and support for branching semantics.

4.1 Model Comparison and Versioning Frameworks

EMF Compare EMF Compare is the standard Eclipse solution for state-based model comparison, identifying differences between model snapshots via matching, difference detection, and merging. Its advantages lie in its mature matching algorithms, which effectively cope with element renamings and structural modifications, and in its seamless integration with the Eclipse ecosystem[28]. Nonetheless, EMF Compare relies solely on state-based comparison and has no access to the underlying operation sequences. This leads to key drawbacks for Vitruvius: the semantics of operations are discarded, meaning that intermediate modifications that ultimately revert to the original state remain undetected, even though they may have triggered consistency rules; causal relationships are not captured, making it impossible to distinguish direct user edits from automatically propagated ones [22]; and fine-grained delta sequences cannot be recovered, which rules out replay and rollback functionalities. Vitruvius already offers comparable state-based comparison with the `StateBasedChangeResolutionStrategy`, which operates on Vitruvius’s `EChange` metamodel. Instead of adding EMF Compare as an external dependency, this work enhances the existing strategy to support branch-sensitive delta computation while reusing established matching heuristics.

EMF Diff/Merge EMF Diff/Merge extends EMF Compare with enhanced three-way merging capabilities, using a specialized algorithm to compare models against a common base. It supports both two-way and three-way comparison and provides extensible mechanisms for detecting and resolving conflicts at the model element level. `DiffMerge` operates on model states and focuses on computing differences between complete models rather than tracking their evolution over time[5]. Consequently, it does not provide facilities for version history management, branch representation, or delta persistence. Despite these limitations, EMF Diff/Merge offers reusable mechanisms for state-based comparison and conflict detection, which may complement operation-based merging approaches.

CDO CDO is a client-server framework offering persistent storage, transactions, and native branching, with strengths in scalability, transactional guarantees, and branch management with temporal queries[20]. However, it conflicts with Vitruvius’s architecture: CDO requires all models in its repository, replacing V-SUM, the core abstraction for consistency. Its persistence model prioritizes query performance over the atomic EChange sequences Vitruvius relies on, and integrating CDO would complicate consistency preservation and require re-engineering the ChangePropagationSpecification. Adoption also entails significant overhead, including migrating from XMI to databases, implementing metamodel adapters, and maintaining server infrastructure - a full framework reimplement rather than a simple extension. Seybold et al.[24] note CDO’s steep learning curve and limited support for model versioning and co-evolution.

EMFStore EMFStore[17] implements operation-based versioning, capturing modifications as sequences of operations instead of full state snapshots. Although this operation-centric paradigm is conceptually similar to Vitruvius’s change tracking, EMFStore, similar to CDO, demands replacing the existing repository infrastructure, and its operation model is incompatible with Vitruvius’s EChange metamodel, making translation layers necessary. In addition, EMFStore has been declared end-of-life and is no longer maintained for current EMF versions.

EML The Epsilon Merging Language (EML) is a rule-based language for defining custom model merge strategies on top of EMF models. EML enables developers to specify how elements from two or more input models should be matched, merged, or discarded, and supports fine-grained, domain-specific conflict resolution logic. Its main strength lies in its expressiveness and flexibility, allowing merge behavior to be tailored precisely to specific modeling languages or project conventions[18][25]. However, EML operates purely on model states and assumes that input models are provided as complete snapshots. It does not offer facilities for version history management, branch tracking, or operation recording. Furthermore, EML requires manual specification of merge rules, which introduces additional modeling effort and shifts responsibility for correctness to the merge definition itself. In the context of Vitruvius, EML is therefore not suitable as a primary merge mechanism. Nevertheless, its rule-based merge concepts provide valuable inspiration for defining configurable and extensible conflict resolution strategies on top of Vitruvius’s operation-based merge infrastructure.

AMOR AMOR represents a significant research contribution, introducing sophisticated conflict detection that combines operation-based and state-based analysis[5]. Its main contribution is a comprehensive conflict taxonomy defining atomic conflict patterns such as delete-delete, update-update, and delete-reference. In addition, AMOR supports composite operation detection, recognizing high-level refactorings from atomic change sequences to enable semantic conflict analysis. While the framework’s theoretical foundations are sound, its implementation is no longer maintained and does not cover consistency-preservation-specific conflicts unique to Vitruvius. This work builds on AMOR’s concepts: atomic conflict patterns form the basis of syntactic conflict detection, composite operation detection informs

future extension points, and conflict prioritization enables automatic resolution of trivial cases. These ideas are extended with Vitruvius-specific semantic conflicts to integrate fully with consistency-preservation rules.

Difference Metamodel Approach The Difference Metamodel approach from Cicchetti et al. [9] represents model differences via an explicit metamodel with first-class types for changes, enabling tool-independent analysis but introducing overhead in metamodel maintenance, serialization, and tool handling. Vitruvius already provides an explicit change metamodel, the EChange hierarchy, representing atomic operations. Instead of adding a separate difference metamodel, this work extends EChange with branch-aware metadata (branch UID, timestamp, author), achieving structured, explicit change representation while maintaining architectural consistency and avoiding the complexity of parallel metamodels.

Efficient Model Matching Similarity-based model matching constitutes the foundation of state-based differencing and merging approaches, but its computational complexity grows cubically with the number of model elements, making it a performance bottleneck for large models. He et al. [13] address this limitation by exploiting the observation that consecutive model versions typically modify only small portions of a model. Their approach accelerates similarity-based matching by identifying equivalent or highly similar subtrees, allowing unchanged regions to be matched efficiently using hash-based techniques. Experimental results show substantial performance improvements while often reducing matching errors. Although this work focuses on optimizing matching algorithms rather than version management or branching semantics, it highlights the importance of scalable model comparison as a prerequisite for efficient delta computation and merging. These insights underpin the design goals of this thesis, which assumes efficient matching mechanisms and focuses instead on integrating branch-aware versioning and merge infrastructure into Vitruvius.

Graph-Modification-Based Model Versioning Taentzer et al. [27] formalize model versioning by representing models as typed attributed graphs and expressing differences between versions as graph modifications derived from pushout constructions in category theory. The approach provides precise definitions for three-way model merging, conflict detection, and conflict resolution at the level of individual graph elements, and is accompanied by a prototype implementation that validates the formal concepts in practice. Its language-agnostic graph foundation is a notable strength, as any modeling language expressible as a typed graph can in principle be versioned with the same formalism. However, the approach operates entirely on model states and encodes differences as structural graph changes, without recording the operation sequences that produced them. Consequently, it cannot distinguish user-authored modifications from automatically propagated ones, which is the provenance distinction that consistency-aware merging in Vitruvius requires [22]. This thesis draws on the conflict classification concepts from Taentzer et al. as a theoretical reference, while replacing the state-level graph-modification layer with Vitruvius's operation-based EChange metamodel to preserve operation provenance and enable replay-based merge.

Formal Foundations for EMF Model Merging Westfechtel [31] provides formal foundations for three-way merging of EMF models, defining merge semantics in terms of a least common ancestor model and difference operations over model elements. The work specifies precisely when a three-way merge is well-formed, what constitutes a conflict between two diverged branches, and under which conditions a conflict-free result can be computed automatically. By grounding merge semantics in the EMF metamodel hierarchy, the formalization is precise and practically relevant to the Eclipse modeling ecosystem. Despite this precision, the approach operates on model states and computes differences by comparing snapshots of the ancestor and branch versions. It therefore shares the fundamental limitation of other state-based techniques: automatically propagated changes are indistinguishable from user-authored ones, so the formalism offers no mechanism to suppress false-positive conflicts arising from consistency rule application[22]. The merge formalization developed in this thesis complements Westfechtel's foundations by lifting the merge operator from the state level to the operation level, where provenance metadata carried by each EChange record allows consequential changes to be filtered before conflict detection begins.

Multi-Version Models and Merge Conflict Detection Barkowsky and Giese [4] address merge conflict detection in the context of multi-version models, where a model may exist in several concurrent versions that must eventually be reconciled. Their approach represents models as typed graphs and detects conflicts by checking whether the combined graph modifications from two branches violate well-formedness constraints of the underlying metamodel. A key strength is that well-formedness checking is integrated directly into conflict detection, so structurally invalid merge results are identified before they are ever produced. The work is, however, focused on structural graph properties and operates on model states rather than recorded operation histories. As a result, it does not account for changes that arise automatically from consistency rules, and it provides no mechanism to distinguish those consequential changes from original user edits[22]. This thesis addresses the same multi-version scenario but replaces structural graph analysis with an operation-based approach, using the provenance information recorded in Vitruvius's changelog to avoid reporting conflicts on changes that would be regenerated automatically during a consistency-aware merge.

Change-Based Model Differencing Yohannis [33] proposes a change-based approach to model differencing and conflict detection in which the sequence of changes applied to a model is recorded persistently rather than recomputed by comparing snapshots. By replaying or comparing these recorded change sequences, the approach can detect conflicts more directly than state-based tools and avoids the matching step that makes snapshot comparison expensive for large models. The work demonstrates that retaining the full change history yields richer conflict information, since the order and nature of edits are preserved alongside the resulting model state. The approach is, however, designed for single-model scenarios and does not consider settings where multiple related models are kept consistent through automated propagation rules. In particular, it does not distinguish between changes that a user authored directly and changes that were produced automatically as a consequence of consistency preservation, which is the distinction at the heart of this thesis[22]. This work builds on the same intuition that recording changes is preferable to

recomputing differences, and extends it to a multi-model setting where provenance metadata separates original from consequential changes before conflict detection takes place.

Domain-Specific Conflict Resolution Zadahmad Jafarlou and Syriani [14] present DSMCompare, a framework for domain-specific conflict resolution and model merging that allows developers to define merge rules and conflict resolution strategies tailored to a particular modeling language. Rather than applying a single generic merge algorithm, DSMCompare lets users encode domain knowledge about which element combinations are compatible and how disagreements between branches should be resolved. This flexibility makes the tool more expressive than general-purpose model merge tools for cases where the semantics of the modeling language are well understood. The approach is nonetheless state-based: merge rules operate on model snapshots and compare element states across branches without access to the history of operations that produced them. As a result, conflicts arising from automatically propagated changes are not distinguished from those caused by direct user edits[22], and domain rules must be written manually to compensate for knowledge that a provenance-aware system could derive automatically. In this thesis, semantic awareness is achieved not through manually defined rules but through the provenance metadata carried by each recorded EChange, which allows consequential changes to be identified and excluded from conflict detection without additional configuration.

4.2 Pointer-Based File Storage: Git

Git is a widely used distributed version control system for source code, providing a reference point for lightweight, pointer-based branching and efficient delta storage. Git stores all repository data in a local `.git/` directory using plain files rather than a database. Branches are represented as simple pointers to commits, and the current branch is tracked via a HEAD reference. Commits themselves store content objects (blobs, trees) and metadata such as author, timestamp, and parent pointers, while merges create new commits linking multiple parents. Git's storage model relies on deltas and snapshots, using hashes for integrity and efficient retrieval, which enables rollback, branching, and merging without a central database[29][10]. This decentralized, file-based approach inspired the proposed `.vitruvius/` storage concept, where model versions, deltas, branch pointers, and snapshots are stored similarly, enabling branch-sensitive versioning and conflict detection while remaining compatible with Vitruvius's operation-based change tracking.

4.3 Comparison of Approaches

Table 4.1 summarizes the approaches discussed in this chapter along six dimensions that are central to the design goals of this thesis: the change representation used, whether branching is natively supported, whether multi-model scenarios are addressed, whether conflict detection is provided, whether the approach distinguishes original from automatically propagated

changes (provenance-awareness), and whether the merge strategy accounts for cross-model consistency rules[22].

Table 4.1: Comparison of model versioning and merge approaches. The Provenance-Aware and Consistency-Aware dimensions are motivated by [22]. ✓ = supported, × = not supported.

Approach	Change Repr.	Branching	Multi-Model	Conflict Detection	Provenance-Aware	Consistency-Aware
EMF Compare [28]	State	×	×	✓	×	×
EMF Diff/Merge [5]	State	×	×	✓	×	×
CDO [20]	State	✓	×	✓	×	×
EMFStore [17]	Operation	×	×	✓	×	×
EML [18]	State	×	×	✓	×	×
AMOR [5]	Hybrid	×	×	✓	×	×
Difference Metamodel [9]	Explicit	×	×	×	×	×
Taentzer et al. [27]	Graph-mod.	×	×	✓	×	×
Westfechtel [31]	State	×	×	✓	×	×
Barkowsky & Giese [4]	Graph-mod.	✓	×	✓	×	×
Yohannis [33]	Operation	×	×	✓	×	×
DSMCompare [14]	State	×	×	✓	×	×
This Thesis	Operation	✓	✓	✓	✓	✓

5 Evaluation

We evaluate the branch-aware vsUM extension through two complementary activities. We first structure the evaluation using the Goal-Question-Metric (GQM) framework, deriving six evaluation questions from a single overarching goal and pairing each question with a concrete, observable metric. We then address these questions through functional tests that verify each subsystem of the implementation in isolation, and through a case study that validates the system end to end in the vehicle braking system domain, exercising four representative collaboration scenarios covering clean merge, conflict detection and resolution, versioning and rollback, and branch lifecycle management. The chapter closes with a discussion of threats to validity.

5.1 Evaluation Goals and Research Questions

We structure the evaluation using the Goal-Question-Metric (GQM) framework [8]. Originally developed in the context of software engineering measurement, GQM provides a principled way to connect high-level evaluation objectives to concrete, observable evidence. The framework operates at three levels. At the top level, a *goal* defines the purpose of the evaluation, the object under study, the quality focus of interest, and the perspective from which it is assessed. Each goal is then refined into a set of *questions* that identify what must be known in order to determine whether the goal is satisfied. A question typically asks about a specific property of the system or process under evaluation. Finally, each question is associated with one or more *metrics*, which are concrete, observable values that can be collected and inspected to answer the question. This three-level hierarchy ensures that every measurement activity is directly traceable to a stated evaluation goal, and that no data is collected without a clear purpose. It also makes the evaluation structure explicit and reproducible: another evaluator following the same questions and metrics would arrive at comparable conclusions.

Goal. Evaluate the branch-aware vsUM extension for the purpose of assessing functional correctness with respect to four identified collaboration scenarios from the point of view of a model-based systems engineer working in a multi-developer team in the context of a safety-critical automotive domain.

From this goal we derive six evaluation questions. Each question is paired with a concrete, observable metric that can be verified by inspecting in-memory model state or method return values after a test step. Table 5.1 lists the questions and their corresponding metrics.

Table 5.1: GQM evaluation questions and metrics.

Question	Metric
Q1: Does the system correctly merge non-overlapping changes without data loss?	All model elements from both branches present in the merged model; merge status SUCCESS.
Q2: Does the system detect when the same attribute is modified on two branches?	Conflict descriptor contains the element UUID and feature name; merge status CONFLICTING.
Q3: Does THEIRS resolution produce a deterministically correct merged model?	Merged model contains the source-branch value; merge status SUCCESS.
Q4: Can a version snapshot be created and used to restore a previous model state?	Model state after rollback matches the state at version creation.
Q5: Are branch model states isolated across checkout operations?	Elements added on branch B are absent from branch A after switching back.
Q6: Does the branch metadata lifecycle transition correctly?	Metadata reflects the expected state and maturity after each operation.

We address these six questions through two complementary evaluation activities. The functional tests, implemented as JUnit test classes in the `vsum/branch` and `vsum/versioning` packages of the Vitruv repository, verify each subsystem in isolation. They cover all six questions at the unit and integration level by exercising individual managers, handlers, storage components, and utility classes with controlled inputs and inspecting their outputs directly. The case study, implemented in the `Vitruvius-Branching-Test` module, validates the same questions at the system level by executing four collaboration scenarios end to end in a realistic multi-model environment. The two activities thus provide both fine-grained component-level coverage and end-to-end confidence that the system behaves correctly under realistic usage conditions.

All six questions are answered positively by the evaluation. Q1 is addressed by the merge handling tests and confirmed by Scenario S1 (Section 5.3.3.1), which demonstrates that non-overlapping changes from two branches are integrated without data loss. Q2 is addressed by `SemanticConflictDetectorTest` and confirmed by Scenario S2 (Section 5.3.3.2), which shows that a diverging attribute modification on the same element UUID is detected and reported before any file-level operation is attempted. Q3 is addressed by `SemanticMergeEngineTest` and confirmed by the resolution phase of S2, which verifies that THEIRS resolution produces a deterministically correct merged model. Q4 is addressed by the versioning tests and confirmed by Scenario S3 (Section 5.3.3.3), which demonstrates that a named version snapshot can be created and used to restore the model to a precise earlier state. Q5 and Q6 are addressed by `BranchManagerTest` and confirmed by Scenario S4 (Section 5.3.3.4), which

verifies that branch model states are correctly isolated across checkout operations and that branch metadata transitions through the expected lifecycle states.

5.2 Functional Tests

The functional tests verify the correctness of individual components of the branch-aware vsum extension. Each test class targets a specific subsystem and exercises it with controlled, isolated inputs, asserting on the outputs directly without involving the full system stack. The tests are organised across two packages in the Vitruv repository: `vsum/branch`, which contains test classes for branching, commit management, conflict detection, merge handling, event handlers, and file system utilities, and `vsum/versioning`, which contains tests for the versioning subsystem. In total, the functional test suite comprises approximately 300 test cases across 24 test classes.

5.2.1 Branch Management and Commit Operations

`BranchManagerTest` covers the full lifecycle of a branch from creation to deletion. It verifies that creating a branch from an existing source persists the correct metadata, including the parent branch name, initial state, and maturity level. The test class further confirms that switching to a non-existent branch raises an appropriate exception and that deletion removes the Git reference while updating the metadata accordingly. The branch topology, which records parent-child relationships independently of Git's own branch listing, is tested under both single and multiple child configurations. State and maturity transitions are also covered, confirming that a branch progresses correctly through its lifecycle states and that external branches receive the expected default values. This test class provides direct coverage for Q5 and Q6.

`CommitManagerTest` verifies that the commit operation stages all model files of registered extensions, writes the post-commit trigger file, and updates the branch metadata timestamp. It also confirms that the operation raises an appropriate error when called with nothing to commit or a blank message. This behaviour is a precondition for Q1 through Q3, since every developer action that modifies a model must be durably recorded in the Git history before conflict detection and merge resolution can operate correctly.

5.2.2 Changelog Storage and Change Recording

The changelog storage tests cover the classes responsible for capturing, classifying, and persisting semantic change entries. `EChangeToEntryConverterTest` verifies that every category of `EChange` notification, including element creation, element deletion, attribute assignment, and reference modification, is correctly mapped to a typed `SemanticChangeEntry` carrying the affected element UUID, the feature name, and the old and new values. `SemanticChangeBufferTest` confirms that the in-memory buffer accumulates changes

across propagation cycles, correctly separates entries by resource URI, and returns an empty, unmodifiable snapshot after draining. The classification and serialisation of individual entries are covered by `SemanticChangeTypeTest` and `SemanticChangeEntryTest`. Finally, `SemanticChangelogManagerTest` verifies that completed changelogs are written to and retrieved from the Git object store using the full commit SHA as the storage key. Since the per-branch changelog is the primary input to conflict detection, the correctness of these components is a prerequisite for Q1 and Q2.

5.2.3 Event Handlers and Background Watchers

The git hook handlers and background watcher threads form the reactive layer of the system, and their test classes verify both the correctness of individual handler logic and the lifecycle behaviour of the long-running watcher threads. `PreCommitHandlerTest` verifies that the handler correctly validates the loaded resources in the virtual model, passing when no errors are present, failing when a resource carries load errors, and handling edge cases such as an empty resource set or a null correspondence model. `PostCommitHandlerTest` confirms that the handler accepts valid branch and commit SHA inputs and rejects null arguments. `PostCheckoutHandlerTest` verifies that the handler rejects null branch names and null virtual model references. `PostMergeHandlerTest` verifies that the handler correctly reloads the vsUM state from the Git object store after a merge checkout, which is the mechanism that ensures the working directory and the in-memory model are consistent following a merge operation. These tests contribute to Q1 and Q5 by confirming that each hook transition leaves the system in a well-defined state.

The three background watcher tests share a common structure: each verifies that the watcher thread starts and stops cleanly, rejects a second start attempt, and terminates within a reasonable time. Beyond lifecycle behaviour, `VsumReloadWatcherTest` confirms that the watcher detects a reload trigger file and performs the reinitialization, handles multiple sequential reinitializations correctly, and remains idle in the absence of a trigger. `VsumValidationWatcherTest` verifies that the watcher writes result files after detecting a validation trigger, produces a success result for a valid model, and continues running after a validation error rather than crashing. `VsumPostCommitWatcherTest` confirms that the post-commit trigger file is detected and cleared correctly. `VsumMergeWatcherTest` verifies the equivalent lifecycle for the merge trigger. The correct behaviour of these watchers is essential for Q1 and Q5, as they are the mechanism by which asynchronous git events drive vsUM state transitions.

5.2.4 File System Utilities and Git Hook Installation

The utility test classes verify the supporting infrastructure that the rest of the system depends on. `GitHookInstallerTest` covers the full installation surface: it confirms that each individual hook script is written with the correct content, that an existing hook is backed up before being replaced, that `.gitattributes` is written correctly for the merge

driver and updated idempotently when the file already exists, and that the merge driver configuration file and the Git configuration entry are created. It also verifies that `installAll` is idempotent and that uninstalling a hook restores the backup when one exists.

The trigger file and result file tests cover the file-based communication protocol between the git hooks and the background watcher threads. `ReloadTriggerFileTest` verifies that a trigger file is created with the correct field format, that each trigger carries a unique request identifier, that the file is atomically consumed and deleted on read, and that malformed field counts and invalid timestamps are handled gracefully. `ValidationResultFileTest` and `ValidationTriggerFileTest` apply the same rigour to the validation protocol, verifying round-trip serialisation of all outcome values through the JSON result file and confirming that result files from different request identifiers remain independent. These tests underpin Q5 and Q6 by ensuring that the file-based signalling between hooks and watchers is robust under normal and error conditions.

5.2.5 Conflict Detection and Merge Handling

`SemanticConflictDetectorTest` systematically exercises the conflict analysis pipeline across a broad range of scenarios. It verifies the absence of conflicts when two branches modify disjoint elements, when both branches record identical changes, and when no changelogs exist for a branch. On the positive side, it confirms that a diverging attribute modification on the same element UUID is classified as a medium-severity conflict and that a delete-versus-modify combination on the same element is classified as high severity. Tombstone detection, which flags modifications to children of a deleted container as implicit conflicts, is tested in both directions and verified to avoid false positives when the element UUIDs do not overlap. The auto-resolution path, which resolves conflicts where one side recorded an original change and the other recorded a consequential change, is also covered. These tests provide the primary evidence for Q2.

`MergeManagerTest` verifies the orchestration layer above the conflict detector. It confirms that a fast-forward merge succeeds and marks the source branch as merged, that a non-fast-forward merge produces a two-parent merge commit, and that a conflicting merge is blocked without marking the branch. The test class also verifies that the merge trigger file is written on success and withheld on conflict, and that guard conditions such as self-merge and merge from a non-existent branch are rejected. This class addresses Q1, Q2, and Q3 at the orchestration level. `SemanticMergeEngineTest` complements this by verifying that the engine correctly replays changelog entries through the vsuM to produce the resolved model state, providing targeted coverage for Q3. `GitMergeDriverTest` verifies the cache staleness checks and cache cleanup operations of the Git merge driver, which is the command-line entry point registered in `.gitattributes`.

5.3 Case Study

We validate the system by executing the four collaboration scenarios defined in the running example (Section 3.2) end to end, using the vehicle braking system domain as a realistic, safety-critical engineering context in which multiple developers work concurrently on overlapping model aspects. While the functional tests verify individual components in isolation, the case study addresses all six evaluation questions at the system level. The case study comprises four scenarios, each targeting one or more of the evaluation questions defined in Section 5.1.

5.3.1 Case Study Domain

The case study uses the vehicle braking system domain introduced as the running example in Section 3.2: the **BrakeSystem** (M1) and **CAD_Model** (M2) metamodels maintained in bidirectional consistency by Vitruvius reaction rules. The common ancestor for all four scenarios is the shared starting state from Section 3.2, a single `disk1` instance with `diameterInMM=120` and `brakeDiskThicknessInMM=30` on the main branch. We refer to Section 3.2 for the full metamodel descriptions, the bidirectional CPR definitions, and the rationale for choosing this domain. The domain is well suited for validation because it combines two model types with non-trivial consistency rules and concrete numeric attributes whose values can be precisely verified before and after each operation.

5.3.2 Test Setup and Methodology

All four scenarios are implemented as `@Test` methods in the `BrakeCaseStudyIntegrationTest` class in the `Vitruvius-Branching-Test` module. Each test initialises a real JGit repository in a `JUnit @TempDir`, constructs a `BranchAwareVirtualModel` with both propagation specifications registered, and sets the change propagation mode to `TRANSITIVE_CYCLIC` so that every model change is bidirectionally propagated through the full reaction chain. Since the tests exercise the `vsuM` programmatic interface directly, four background watcher threads are started to handle validation, reload, merge, and post-commit events respectively.

Model operations are performed through the `vsuM` view API using direct `EChange` notification recording. Every structural change, such as attribute assignments and element insertions, produces `EChange` notifications that carry the affected element's stable UUID. Changes are committed via `CommitManager`, which generates branch-scoped semantic changelog entries from the recorded `EChange` stream. Branch, merge, and versioning operations are invoked through `BranchManager`, `MergeManager`, and `VersioningService` respectively. Assertions verify model-level attribute values, merge result status, conflict descriptors, and branch metadata after each operation.

5.3.3 Scenario Results

We present four scenarios that together cover all six evaluation questions. Each scenario describes the initial setup, the sequence of operations performed, and the observed result. The mapping to evaluation questions is stated at the end of each scenario.

5.3.3.1 S1: Clean Merge

Setup. Starting from a common ancestor on main, a Brakesystem root is created and a BrakeDisk"disk1" is added with diameterInMM=120. The creation propagates to M2: a Namespace"disk1" appears in the CAD_Model with a NumericParameter"Diameter"=120.0. A branch feature-diameter is created from this state.

Execution. On feature-diameter, disk1.diameterInMM is changed from 120 to 130. The corresponding reaction fires and updates Parameter"Diameter" in M2 to 130.0. On main, a new ABSensor"sensor1" is added and the reaction creates a corresponding Namespace"sensor1" in M2. Both changes are committed and their changelog entries are stored in the Git object store. At merge time, the SemanticConflictDetector compares both changelogs: feature-diameter records an ATTRIBUTE_CHANGED entry for disk1.diameterInMM, while main records an ELEMENT_CREATED entry for sensor1. The two streams share no common element UUID and feature pair, so no conflict is detected and the merge proceeds.

Result. The merge completes with status SUCCESS. The merged model state reflects the contributions of both branches: disk1.diameterInMM equals 130, sensor1 is present in the Brakesystem, Parameter"Diameter" in M2 holds the value 130.0, and Namespace"sensor1" is present in the CAD_Model. The result demonstrates that the system preserves the intent of both developers without data loss and maintains bidirectional M1-M2 consistency across the merge. Conflict absence is determined at the semantic level through UUID-keyed changelog comparison, independently of how the changes happen to be serialised in the model files. This scenario provides a positive answer to Q1.

5.3.3.2 S2: Conflict Detection and Resolution

Setup. Starting from the same common ancestor as S1, a branch conflict-diameter is created. On main, disk1.diameterInMM is changed from 120 to 130. On conflict-diameter, the same attribute is changed from 120 to 350. Both changes are committed on their respective branches.

Execution: Conflict Detection. A merge is initiated without a conflict resolution strategy. The `SemanticConflictDetector` reads both changelogs and identifies that both branches record an `ATTRIBUTE_CHANGED` entry for the same element UUID and the same feature `diameterInMM`, but with different target values. This is classified as a direct attribute conflict. The merge is blocked before any file-level operation is performed and the model files remain free of conflict markers. The merge result carries status `CONFLICTING` and includes the conflict descriptor `semantic://{disk1-uuid}/diameterInMM`, providing a precise, position-independent pointer to the conflicting element and feature.

Execution: Conflict Resolution. The merge is re-initiated with a `THEIRS` resolution strategy. The `SemanticMergeEngine` replays the conflict-diameter changelog entries through the `vsun`: `disk1.diameterInMM` is set to 350, which triggers the reaction that updates `Parameter"Diameter"` in M2 to 350.0. A two-parent merge commit is created and the `vsun` is reloaded from the merged state.

Result. The merge completes with status `SUCCESS`. The merged model state reflects the source branch version: `disk1.diameterInMM` equals 350 and `Parameter"Diameter"` in M2 holds the value 350.0. The model files contain no conflict markers. The scenario demonstrates that the system correctly identifies a semantic conflict at the element-feature level before any file is touched, and that `THEIRS` resolution produces a deterministically correct merged state while preserving bidirectional M1-M2 consistency. This scenario provides a positive answer to Q2 and Q3.

5.3.3.3 S3: Versioning and Rollback

Setup. This scenario begins from the post-merge state of S1, where `disk1` with `diameterInMM=130` and `brakeDiskThicknessInMM=30`, and `sensor1` are present on main. Both M1 and M2 are in a consistent state.

Execution. A named version `v1.0` is created via `VersioningService`, which anchors the version to the current `HEAD` commit and stores version metadata in the repository. Subsequently, `disk1.brakeDiskThicknessInMM` is changed from 30 to 25 and committed. Before performing the rollback, a preview is requested. The preview returns the list of commits that would be abandoned, giving the developer full transparency over the scope of the operation before it is applied. The rollback is then confirmed: the `HEAD` is reset to the `v1.0` commit and the `vsun` is reloaded from the restored model files.

Result. After rollback, the model state reflects the version point exactly. `disk1.brakeDiskThicknessInMM` is restored to 30 and `Parameter"BrakeDiskThickness"` in M2 holds the value 30.0. `sensor1` remains present, as it was committed before the version point and is unaffected by the rollback. The preview returned exactly the expected number of commits to be abandoned, demonstrating that the version anchor is backed by an immutable commit reference and that the rollback scope is both predictable and transparent. This scenario provides a positive answer to Q4.

5.3.3.4 S4: Branch Lifecycle

Setup. Starting from main with disk1 and sensor1 present in both M1 and M2, a branch feature-caliper is created via BranchManager. The returned metadata records the branch name, parent branch main, state ACTIVE, and maturity DRAFT. The vsUM switches to the new branch.

Execution. On feature-caliper, a BrakeCaliper"caliper1" is added to the Brakesystem. The reaction fires and creates a corresponding Namespace"caliper1" in the CAD_Model. The branch is then switched back to main and the vsUM is reinitialized. A query through the vsUM view finds no element with the identifier caliper1 in either M1 or M2, confirming that the element exists only on feature-caliper and is not visible from main. The branch topology is queried and returns the entry main->[feature-caliper], confirming that the system tracks parent-child relationships independently of Git's own branch listing. The branch is then deleted and the branch list no longer contains an active entry for feature-caliper.

Result. The scenario confirms two independent properties of the system. First, model state is correctly isolated between branches: elements introduced on feature-caliper are not visible from main in either M1 or M2 after switching back, demonstrating that branch checkout replaces the full vsUM state rather than only the tracked model files. Second, the branch metadata lifecycle behaves correctly: the topology reflects the parent-child relationship immediately after branch creation, and the branch is fully removed from the active branch listing after deletion. This scenario provides a positive answer to Q5 and Q6.

5.4 Threats to Validity

We identify four categories of threats to the validity of the evaluation following the classification of Wohlin et al. [32]

Internal Validity. The four scenarios cover concrete collaboration patterns but represent a small subset of possible multi-developer workflows. In particular, each scenario exercises at most two concurrent branches. The dependency graph construction and topological sorting in the SemanticConflictDetector are not evaluated under histories with many concurrent branches or long chains of consequential changes. Similarly, the rollback scenario uses a two-commit history since the version point and does not assess behaviour under longer histories.

Construct Validity. The test scenarios exercise only a subset of the brake system metamodel: only BrakeDisk, ABSensor, and BrakeCaliper are used. Further attributes such as rimHoleNumber and ventilated on BrakeDisk, or multi-valued cross-references between components, are not covered. Each scenario modifies at most one attribute of one element at a time, so the multi-feature conflict resolution and the full dependency graph replay in steps five to eight of the SemanticConflictDetector are not exercised. The mapping from test outcomes to the GQM metrics is direct and based on observable in-memory model state, which reduces instrumentation bias.

External Validity. The case study is conducted in a single domain. The core mechanisms under evaluation, including EChange recording, UUID-based conflict detection, JGit object store access, and xMI serialisation, are metamodel-agnostic and not specific to the braking domain. However, the study does not evaluate behaviour in the presence of complex metamodel features such as multi-valued cross-references, derived features, or deep inheritance hierarchies. Validation against additional domains and more complex metamodels is identified as future work.

Scalability. The case study uses small models containing two to three elements and short branch histories of two to three commits per branch. The performance of changelog lookup via JGit RevWalk, conflict graph construction, and vsuM reload under large models or long histories is not measured. Performance evaluation is outside the scope of the current validation and is left as future work.

6 Conclusion and Future Works

6.1 Conclusion

This thesis has presented the design and implementation of a branch-aware model versioning extension for the Vitruvius framework. The extension addresses a fundamental limitation of the existing system: the inability to support parallel, isolated development lines over a shared virtual model. By introducing branching and versioning as first-class operations, the framework becomes applicable to the multi-developer workflows that characterise real-world model-based systems engineering. The implementation is structured around four interdependent subsystems: branch lifecycle management, semantic change recording, merge handling with conflict detection and resolution, and model state versioning. A REST API layer exposes all subsystem operations programmatically, enabling integration with external tooling. The evaluation demonstrates that the system meets all six stated evaluation goals, as verified by a functional test suite of approximately 300 test cases and a case study conducted in the vehicle braking system domain.

The branch lifecycle subsystem provides explicit management of branch creation, switching, deletion, and topology tracking through `BranchManager`, with metadata persisted in the Git object tree so that branch context survives checkout operations. The semantic change recording subsystem captures every model modification as a typed, UUID-keyed `SemanticChangeEntry` and stores the resulting changelog durably in the Git object store, with each changelog file named after the abbreviated commit SHA to establish a direct correspondence with `git log` output. The merge handling subsystem uses these changelogs to detect conflicts at the semantic level before any file-level merge is attempted, classifying conflicts by severity and supporting automatic resolution of original-versus-consequential change pairs. When no conflicts remain, the `SemanticMergeEngine` replays the source branch changelog through the `vsuM` to produce a merged model state that respects bidirectional consistency rules. The versioning subsystem anchors named snapshots to Git annotated tags and provides a two-step rollback mechanism that previews the commits to be abandoned before the HEAD is reset, giving developers explicit control over the scope of the operation.

6.2 Limitations

The system operates under a set of operational constraints that developers must be aware of before use. The four Git hooks that drive `vsuM` synchronisation must be installed manually via `GitHookInstaller.installAllHooks()` before any branch operation is performed

through the command line, as the hooks are not installed automatically on repository initialisation. For the same reason, every developer who clones the repository must run this installation step locally, since Git does not transfer the contents of `.git/hooks/` on clone or fetch. The system additionally requires the repository to contain at least one commit before the branching subsystem can be initialised, as several internal operations depend on a resolvable HEAD reference. All watcher threads that consume hook trigger files are in-process daemon threads, meaning the Java process must remain running for the duration of any CLI-triggered operation. Finally, new branches must be created via `git checkout -b` or `git switch -c` rather than `git branch`, as only an actual working-tree switch triggers the post-checkout hook that initialises branch metadata and inherits the parent vsUM state.

Beyond the operational constraints, the system has several behavioral boundaries that define its current scope. The vsUM reinitialization triggered by a CLI branch checkout is asynchronous: the post-checkout hook writes a trigger file and exits, and the `VsumReloadWatcher` consumes it on its next poll cycle, leaving a short window during which the working tree and the in-memory model state may be out of sync. The installed hook scripts depend on a POSIX-compatible shell interpreter. On Linux and macOS this is satisfied natively; on Windows the hooks execute correctly when Git for Windows is installed, but fail silently when invoked from a plain command prompt or PowerShell session without a Bash interpreter in the path. Branch metadata and topology tracking cover only locally checked-out branches: branches that exist solely as remote-tracking references, most commonly those created through a remote web interface without a local checkout, are not visible to `BranchManager.listBranches()`. The merge subsystem is designed exclusively for cross-branch integration and does not handle the case where two developers commit to the same branch and one pulls the other's changes directly, as this falls outside the branching model the system is built around. Finally, semantic conflict detection relies on UUID-keyed changelog entries produced by Vitruvius's EMF notification pipeline. When changes are submitted through the REST client using the state-based change derivation strategy, element renames appear as a deletion followed by a creation with a new UUID, and the conflict detector therefore does not identify them as modifications to the same element.

Two additional constraints affect the semantic changelog subsystem specifically. The post-commit hook delivers an asynchronous trigger: if the Java process terminates within the 500-millisecond polling window immediately after a direct Git commit, the corresponding changelog entry is never written and cannot be reconstructed retroactively, leaving any branch history analysis incomplete for the affected commit. The reaction change filter in `SemanticChangeBuffer` further depends on an internal propagator behaviour that is not guaranteed by the Vitruvius API: the propagator currently reuses `EChange` instances by reference between the consequential and original fields of adjacent `PropagatedChange` objects, which allows the filter to exclude reaction-produced changes by object identity. A future refactoring that introduces defensive copying would silently invalidate this assumption, causing reaction changes to appear in the changelog without any indication of failure.

6.3 Future Work

Several directions present themselves for extending the current implementation. The most immediate is addressing the state-based change derivation gap in conflict detection. Replacing or augmenting `DefaultStateBasedChangeResolutionStrategy` with an operation-recording approach on the REST client side would allow renames and other composite operations to be represented as modifications to stable element UUIDs, bringing the REST path into full parity with the programmatic API path for conflict detection purposes.

Two further improvements address the robustness of the infrastructure layer. The asynchronous gap introduced by the hook-based reinitialization mechanism could be reduced by adding a readiness signal: after the `VsumReloadWatcher` finishes loading the branch state, it would write a confirmation file that the server checks before handling new requests, ensuring that callers always see a consistent model state. The dependency on a POSIX shell interpreter could be resolved by generating platform-native hook scripts during installation. Providing PowerShell-based hooks for Windows would remove the requirement for Git for Windows and allow the system to run in standard Windows developer environments.

Remote branch visibility and multi-developer workflows on shared branches are two directions with broader architectural implications. Extending `BranchManager` to recognise remote-tracking references would allow branches created through a remote web interface to appear in `listBranches()` without requiring a local checkout. This could be achieved through a server-side push hook that publishes branch metadata to a shared location, or a fetch step that retrieves metadata for newly discovered remote references. Supporting scenarios where two developers contribute directly to the same branch would require a separate pull-based merge strategy that integrates diverging commit sequences on a single branch, rather than across branches, extending the merge subsystem beyond its current scope.

A smaller but useful extension concerns the raw EChange serialization. The semantic changelog already records the name of the affected structural feature as a plain string in each `SemanticChangeEntry`, where it serves as half of the conflict detection key alongside the element UUID. The raw EChange objects produced by the Vitruvius notification pipeline, however, expose the same information only through EMF cross-reference URIs, which require knowledge of the metamodel to interpret. Duplicating the feature name as a plain readable string field directly in the EChange serialization would make the low-level change format self-descriptive for users who process the eChanges payload without going through the semantic changelog layer, at negligible memory cost given that a structural feature name is a short identifier.

Acknowledgements: AI Use We used Gemini (Google) and Claude (Anthropic) during the preparation of this thesis. AI assistance was employed for polishing written text, producing figures and exploring initial ideas for design approaches. The implementation, research questions, system architecture, and all core technical decisions were developed independently by us. All AI-assisted content was critically reviewed and revised before inclusion.

Bibliography

- [1] Juxhin Abazi. *Integrating Maturity into Consistency Preservation*. 2025.
- [2] Kerstin Altmanninger, Martina Seidl, and Manuel Wimmer. “A survey on model versioning approaches”. In: *International Journal of Web Information Systems* 5.3 (2009), pp. 271–304.
- [3] Colin Atkinson and Thomas Kuhne. “Model-driven development: a metamodeling foundation”. In: *IEEE software* 20.5 (2003), pp. 36–41.
- [4] Matthias Barkowsky and Holger Giese. “Towards development with multi-version models: detecting merge conflicts and checking well-formedness”. In: *International Conference on Graph Transformation*. Springer. 2022, pp. 118–136.
- [5] Petra Brosch et al. “An introduction to model versioning”. In: *International school on formal methods for the design of computer, communication and software systems*. Springer, 2012, pp. 336–398.
- [6] Cédric Brun and Alfonso Pierantonio. “Model differences in the eclipse modeling framework”. In: *UPGRADE, The European Journal for the Informatics Professional* 9.2 (2008), pp. 29–34.
- [7] Hugo Bruneliere et al. “A feature-based survey of model view approaches”. In: *Software & Systems Modeling* 18.3 (2019), pp. 1931–1952.
- [8] Victor R Basili¹ Gianluigi Caldiera and H Dieter Rombach. “The goal question metric approach”. In: *Encyclopedia of software engineering* (1994), pp. 528–532.
- [9] Antonio Cicchetti, Davide Di Ruscio, and Alfonso Pierantonio. “Managing model conflicts in distributed development”. In: *International Conference on Model Driven Engineering Languages and Systems*. Springer. 2008, pp. 311–325.
- [10] Mary Rose Cook. *Git From the inside-out*. URL: <https://maryrosecook.com/blog/post/git-from-the-inside-out>.
- [11] Zinovy Diskin et al. “From state-to delta-based bidirectional model transformations: The symmetric case”. In: *International Conference on Model Driven Engineering Languages and Systems*. Springer. 2011, pp. 304–318.
- [12] Nathan Hagel et al. “Explainability in Automated Cross-Domain Model-Driven Brake System Development”. In: *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. IEEE. 2025, pp. 204–211.
- [13] Xiao He et al. “Accelerating similarity-based model matching with subtree equivalence”. In: *Information and Software Technology* (2025), p. 107879.

- [14] Manouchehr Zadahmad Jafarlou and Eugene Syriani. “Domain-specific conflict resolution and model merge”. In: *Journal of Systems and Software* 232 (2026), p. 112694.
- [15] Benedikt Jutz and Thomas Weber. “Scalability of Consistency Preservation with Vitruvius”. In: *Softwaretechnik-Trends Band 44, Heft 4*. Gesellschaft für Informatik eV. 2024.
- [16] Heiko Klare et al. “Enabling consistency in view-based system development—the vitruvius approach”. In: *Journal of Systems and Software* 171 (2021), p. 110815.
- [17] Maximilian Koegel and Jonas Helming. “EMFStore: a model repository for EMF models”. In: *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 2*. 2010, pp. 307–308.
- [18] Dimitrios S Kolovos, Richard F Paige, and Fiona AC Polack. “Merging models with the epsilon merging language (eml)”. In: *International conference on model driven engineering languages and systems*. Springer. 2006, pp. 215–229.
- [19] Tom Mens. “A state-of-the-art survey on software merging”. In: *IEEE transactions on software engineering* 28.5 (2002), pp. 449–462.
- [20] Richard F Paige, Nicholas Matragkas, and Louis M Rose. “Evolving models in model-driven engineering: State-of-the-art and future challenges”. In: *Journal of Systems and Software* 111 (2016), pp. 272–280.
- [21] Jakob Pietron. “Enhancing collaborative modeling”. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. 2020, pp. 1–6.
- [22] Anonymous Authors - Being under review. *Replay-Based Merge for Delta Based Consistency Management in Multi-Model Development*. 2026.
- [23] Anonymous Authors - Being under review. *Replication package: Replay based merge for multi-model consistency networks*. <https://anonymous.4open.science/r/replay-merge-multi-model-models26/README.md>. 2026.
- [24] Daniel Seybold et al. “Experiences of models@ run-time with EMF and CDO”. In: *Proceedings of the 2016 ACM SIGPLAN International Conference on Software Language Engineering*. 2016, pp. 46–56.
- [25] Mohammadreza Sharbaf and Bahman Zamani. “Configurable three-way model merging”. In: *Software: Practice and Experience* 50.8 (2020), pp. 1565–1599.
- [26] Patrick Stünkel et al. “Comprehensive systems: a formal foundation for multi-model consistency management”. In: *Formal Aspects of Computing* 33.6 (2021), pp. 1067–1114.
- [27] Gabriele Taentzer et al. “A fundamental approach to model versioning based on graph modifications: from theory to implementation”. In: *Software & Systems Modeling* 13.1 (2014), pp. 239–272.
- [28] EMFCompare DEV Team. *EMFCompare Homepage*. URL: <https://eclipse.dev/emf/compare/>.
- [29] Git Team. *Git Book*. URL: <https://git-scm.com/book/en/v2>.

-
- [30] Vitruv-Tools. *Vitruv-Server*. <https://github.com/vitruv-tools/Vitruv-Server>. 2026.
 - [31] Bernhard Westfechtel. “Merging of EMF models: Formal foundations”. In: *Software & Systems Modeling* 13.2 (2014), pp. 757–788.
 - [32] Claes Wohlin et al. *Experimentation in software engineering*. Vol. 236. Springer, 2012.
 - [33] Alfa Yohannis. “Change-Based Model Differencing and Conflict Detection”. PhD thesis. University of York, 2020.